ANTI MONEY LAUNDERING SYSTEM IN CRYPTOCURRENCY USING AI&ML

A PROJECT REPORT

Submitted By

STANLY S	**510421104098**
SUBRAMANI S	**510421104100**
SUDHARSHAN S	**510421104102**
VIJAYAKUMAR V	**510421104116**

In partial fulfilment for the award of the degree Of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

ARUNAI ENGINEERING COLLEGE

(AUTONOMOUS)

TIRUVANNAMALAI – 606 603

ANNA UNIVERSITY:: CHENNAI 600 025

MAY 2025

ANNA UNIVERSITY: CHENNAI-600025

BONAFIDE CERTIFICATE






Certified that this project report titled **"ANTI MONEY LAUNDERING SYSTEM IN CRYPTOCURRENCY USING AI&ML"** is the bonafide work done by STANLY S (510421104098), SUBRAMANI S (510421104100), SUDHARSHAN S (510421104102), VIJAYAKUMAR V (510421104116), who carried out the project work under my supervision.

SIGNATURE OF SUPERVISOR

Mr. K.NEELAMEGAM M.Sc., M.Tech

Associate Professor,

Computer Science and Engineering,

Arunai Engineering College,

Tiruvannamalai-606 603.

SIGNATURE OF HOD

Mrs. V.UMADEVI, M.E,

ASP – HEAD OF THE DEPARTMENT

Computer Science and Engineering,

Arunai Engineering College,

Tiruvannamalai-606 603.

INTERNAL EXAMINER

EXTERNAL EXAMINER

CERTIFICATE OF EVALUATION

COLLEGE NAME :5104-ARUNAI ENGINEERING COLLEGE

BRANCH : COMPUTER SCIENCE AND ENGINEERING

SEMESTER : 8th SEMESTER (2021-2025)

SUBJECT CODE & NAME: CS8811-PROJECT WORK

DATE :

NAME OF THE STUDENT	REGISTER NUMBER	TITLE OF THE PROJECT	NAME OF GUIDE WITH DESIGNATION
STANLY S	510421104098	ANTI MONEY LAUNDERING SYSTEM IN CRYPTOCURRENCY USING AI&ML	MR.K.NEELAMEGAM
SUBRAMANI S	510421104100		
SUDHARSHAN S	510421104102		
VIJAYAKUMAR V	510421104116		

The report of the **PROJECT WORK** submitted for the fulfilment of Bachelor of Engineering degree in **COMPUTER SCIENCE AND ENGINEERING** of Anna University was evaluated and confirmed to be report of the work done by the above students.

SIGNATURE OF SUPERVISOR

Mr. K.NEELAMEGAM M.Sc., M.Tech

Associate Professor,

Computer Science and Engineering,

Arunai Engineering College,

Tiruvannamalai-606 603.

SIGNATURE OF HOD

Mrs. V.UMADEVI, M.E,

ASP – HEAD OF THE DEPARTMENT

Computer Science and Engineering,

Arunai Engineering College,

Tiruvannamalai-606 603.

INTERNAL EXAMINER EXTERNALEXAMINER

ACKNOWLEDGEMENT

It is our duty to thank the GOD Almighty and our beloved parents and teachers for their persistent blessing and constant encouragement to reach this level of ,what we are today.

A project of this nature needs co-operation and support from many for successful completion. In this regard ,we are fortunate to express our heart felt thanks to our beloved Founder Chairman **Mr.E.V.VELU**, Chair Person **Mrs.SANKARI VELU** and **Er.E.V.KUMARAN,M.E.,** Vice Chairman, for providing necessary facilities with high-class environment throughout the course.

We take the privilege of expressing our sincere thanks to our beloved Registrar **Dr.R.SATHIYASEELAN, Ph.D.,** for granting permission to undertake the project and we also express our hearty thanks to principal, **Dr.C.ELANCHEZHIAN, Ph.D.,** for their advice in accomplishing this project work.

We are most grateful to **Mrs.V.UMADEVI, M.E.,** ASP-Head of the Department, Department of Computer Science and Engineering, who has given us both moral and technical support adding experience to the job we have undertaken.

We are most grateful to our guide **Mr. K.NEELAMEGAM M.Sc., M.Tech.,** for her astonishing guidance and encouragement for the successful completion of this project. They held us to the highest standards of quality and accuracy.

Last but not least we would like to thank our family members, friends, teaching and non-teaching faculties, who has assisted us directly or indirectly throughout this project.

ABSTRACT

Money laundering poses a significant threat to global financial systems, enabling criminals to disguise illicitly obtained funds as legitimate income. Its detection remains an ongoing challenge for law enforcement agencies, especially as financial transactions increasingly shift toward complex digital ecosystems, including cryptocurrency. Traditional methods of detecting money laundering, such as rule-based systems and manual audits, are proving to be inadequate in the face of sophisticated laundering techniques and the sheer volume of transactions.

This project proposes the development of an AI and machine learning-based Anti-Money Laundering (AML) system designed to meet these modern challenges. The system integrates advanced cryptographic techniques to ensure data privacy and integrity while enhancing the accuracy and efficiency of detecting illicit financial transactions. Machine learning algorithms—such as anomaly detection, clustering, and classification—are employed to analyze large-scale transaction data, identify hidden patterns, and predict anomalies that may indicate money laundering activities. The system also incorporates real-time monitoring capabilities to flag suspicious activities promptly, enabling quicker response and intervention.

By combining the strengths of artificial intelligence, machine learning, and cryptography, the proposed AML system significantly improves detection and prevention mechanisms. This hybrid approach not only enhances regulatory compliance but also provides a scalable and adaptive framework that can evolve alongside emerging financial technologies. The project aims to contribute to the global fight against financial crime by offering a robust, intelligent solution capable of mitigating risks and safeguarding the integrity of financial systems.

ABSTRACT(TAMIL)

ஏ.ஐ மற்றும் எம்.எல் தொழில்நுட்பங்களை பயன்படுத்தி கிரிப்டோகரன்சியில் பணப்பாசறையை தடுக்கும் அமைப்பு

கள்ளப்பணச் சுழற்சி உலகளாவிய நிதி அமைப்புகளுக்கு ஒரு முக்கியமான அச்சுறுத்தலாக இருந்து வருகிறது, ஏனெனில் இது குற்றவாளிகளைத் தங்களின் சட்டவிரோதமாகக் கிடைத்த பணத்தை சட்டப்படி வந்த வருமானமாக மாற்ற உதவுகிறது. குறிப்பாக நிதி பரிவர்த்தனைகள் மேலதிகமாக குறுந்தகவல் தொழில்நுட்பங்களின் நோக்கில், குறிப்பாக கிரிப்டோகரன்சி போன்ற சிக்கலான டிஜிட்டல் சூழல்களுக்கு நகரும் சூழ்நிலையில், இதனை கண்டறிதல் இன்னமும் சிறப்பாக சவாலாக உள்ளது. கள்ளப்பணச் சுழற்சியை கண்டறியும் மரபுக் கூடங்கள், மாதிரிகள் சார்ந்த அமைப்புகள் மற்றும் கைமுறையிலான ஆய்வுகள் போன்றவை, சிக்கலான நவீன சுழற்சி முறைகள் மற்றும் பரிவர்த்தனைகளின் பெரும் அளவுக்கேற்ப போதுமானதாக இல்லாமல் இருக்கின்றன. இந்த திட்டம், இந்த நவீன சவால்களை எதிர்கொள்ளக்கூடிய, செயற்கை நுண்ணறிவு மற்றும் மெஷின் லெர்னிங் அடிப்படையிலான ஒரு எதிர்ப்பு-கள்ளப்பணச் சுழற்சி (AML) அமைப்பை உருவாக்குவதை முன்வைக்கிறது. தரவு பாதுகாப்பும் முழுமையும் உறுதி செய்யும் உயர்ந்த குறியாக்க நுட்பங்களைச் சேர்த்து, சட்டவிரோத நிதி பரிவர்த்தனைகளை கண்டறியும் துல்லியம் மற்றும் செயல்திறனை மேம்படுத்துகிறது. மிகப்பெரிய அளவிலான பரிவர்த்தனை தரவை பகுப்பாய்வு செய்ய, மறைக்கப்பட்ட படிமங்கள் மற்றும் சாத்தியமான சந்தேகத்திற்கிடமான செயல்களை கணிக்க, அசாதாரணம் கண்டறிதல், களங்கப்படுத்தல் மற்றும் வகைப்படுத்தல் போன்ற மெஷின் லெர்னிங் அல்காரிதங்கள் பயன்படுத்தப்படுகின்றன. மேலும், சந்தேகத்திற்கிடமான செயல்களை உடனுக்குடன் கேப்சர் செய்யும் நேரடி கண்காணிப்பு திறனும் இதில் உள்ளடக்கப்பட்டுள்ளது. செயற்கை நுண்ணறிவு, மெஷின் லெர்னிங் மற்றும் குறியாக்கம் ஆகியவற்றின் வலிமைகளை ஒன்றிணைக்கும் இந்த AML அமைப்பு, கண்டறிதல் மற்றும் தடுப்பு செயல்முறைகளை முக்கியமாக மேம்படுத்துகிறது. இந்த ஒருங்கிணைந்த அணுகுமுறை ஒழுங்குமுறை பூர்த்தியை மேம்படுத்துவதோடு, வளர்ந்து வரும் நிதி தொழில்நுட்பங்களுக்கு ஏற்ப மாறக்கூடிய அளவீட்டுத் திறனும் வழங்குகிறது. உலகளாவிய நிதிக் குற்றங்களை எதிர்க்க இந்த திட்டம் ஒரு வலுவான, புத்திசாலியான தீர்வை வழங்குவதற்காக நோக்கமிட்டுள்ளது, இது ஆபத்துகளை குறைத்து நிதி அமைப்புகளின் முழுமையை பாதுகாப்பதற்கும் உதவியாக இருக்கும்.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1	**INTRODUCTION**	**1**
	1.1 INTRODUCTION	**1**
	1.2 AIM OF THE PROJECT	**1**
	1.3 PROBLEM STATEMENT	**1**
	1.4 OBJECTIVE	**1**
	1.5 SCOPE	**2**
	1.6 INCLUSION AND EXCLUSION CRITERIA	**2**
	1.6.1 SUMMARIZING SYSTEM	**3**
	1.6.2 ALGORITHM	**3**
2	**LITERATURE SURVEY**	**5**
3	**SYSTEM ANALYSIS**	**7**
	3.1 EXISTING SYSTEM	**7**
	3.1.1 LIMITATIONS OF THE EXISTING SYSTEM	**7**
	3.2 PROPOSED SYSTEM	**8**
	3.3 ADVANTAGES	**8**
	3.4 DISADVANTAGES	**9**
	3.5 SYSTEM REQUIREMENTS	**9**
	3.5.1 HARDWARE REQUIREMENTS	**9**
	3.5.2 SOFTWARE REQUIREMENTS	**9**
4	**MODULE DESCRIPTION**	**10**
	4.1 DATA COLLECTION	**10**
	4.2 PREPROCESSING	**10**
	4.3 FEATURE EXTRACTING	**10**
	4.4 MODEL TRAINING	**10**

	4.5 CRYPTOGRAPHIC INTEGRITY CHECK	**10**
	4.6 ANOMALY DETECTION	**10**
5	**SYSTEM DESIGN**	**11**
	5.1 PROCEDURAL DESIGN	**11**
	5.2 DATA FLOW DIAGRAM	**11**
	5.2.1 DFD SYMBOLS	**12**
	5.2.2 DATA FLOW	**12**
	5.3 UML DIAGRAM	**14**
	5.4 USE CASE DIAGRAM	**15**
	5.5 SEQUENCE DIAGRAM	**16**
	5.6 CLASS DIAGRAM	**16**
	5.7 ARCHITECTURE DESIGN	**17**
	5.8 SYSTEM DESIGN	**18**
	5.9 NORMALISATION	**18**
	5.9.1 FIRST NORMALISATION TABLE	**19**
	5.9.2 SECOND NORMALISATION TABLE	**19**
	5.9.3 THIRD NORMALISATION TABLE	**19**
6	**SYSTEM DEVELOPMENT**	**20**
	6.1 SYSTEM DESCRIPTION	**20**
	6.2 INTRODUCTION TO PYTHON	**20**
	6.3 LIBRARIES IN PYTHON	**22**
	6.4 WORKING OF PYTHON LIBRARY	**23**
	6.5 PYTHON STANDARD LIBRARY	**23**
	6.6 USE OF LIBRARIES IN PYTHON	**25**
	6.7 DESIGN PHILOSOPHY AND FEATURES	**26**
	6.8 SYNTAX AND SEMANTICS	**28**
	6.9 DEVELOPMENT	**30**

	6.9.1 API DOCUMENTATION GENERATORS	**31**
	6.9.2 NAMING	**31**
	6.10 OPEN CV	**32**
	6.11 OPENCV LIBRARY MODULES	**33**
	6.11.1 CORE FUNCTIONALITY	**33**
	6.11.2 IMAGE PROCESSING	**33**
	6.11.3 VIDEO	**33**
	6.11.4 VIDEO I/O	**33**
	6.11.5 CALIB 3D	**34**
	6.11.6 FEATURES 2D	**34**
	6.11.7 OBJ DETECT	**34**
	6.11.8 HIGH GUI	**34**
	6.12 PILLOW	**34**
	6.13 TENSORFLOW	**35**
	6.13.1 EASY MODEL BUILDING	**35**
	6.13.2 ROBURST ML PRODUCTION ANYWHERE	**35**
	6.13.3 POWERFUL EXPERIMENTATION FOR RESEARCH	**35**
	6.14 TORCH	**36**
7	**SYSTEM TESTING**	**37**
	7.1. TYPES OF TESTS	**37**
	7.1.1 UNIT TESTING	**37**
	7.1.2 INTEGRATION TESTING	**38**
	7.1.3 VALIDATION TESTING	**38**
8	**SOURCE CODE**	**39**
9	**OUTPUT**	**45**
10	**CONCLUSION**	**51**
11	**REFERNCES**	**53**

LIST OF FUGURES

FIGURE NO	NAME	PAGE NO
5.1	PROCEDURAL DESIGN	**11**
5.2	DFD SYSMBOLS	**12**
5.3	DATA FLOW	**13**
5.4	UML DIAGRAM	**14**
5.5	USE CASE DIAGRAMS	**15**
5.6	SEQUENCE DIAGRAM	**16**
5.7	CLASS DIAGRAM	**17**
5.8	ARCHITECTURE DESIGN	**17**

LIST OF ABBREVATION

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
AML	Anti-Money Laundering
UI	User Interface
API	Application Programming Interface
JPEG	Joint Photographic Experts Group
PNG	Portable Network Graphics
BMP	Bitmap Image File
TIFF	Tagged Image File Format
PIL	Python Imaging Library
OS	Operating System
IDE	Integrated Development Environment
C++	C Plus Plus
CPU	Central Processing Unit
GPU	Graphics Processing Unit
I/O	Input/Output
JSON	JavaScript Object Notation
XML	eXtensible Markup Language
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Anti-money laundering (AML) efforts have traditionally relied on rules-based systems, but these struggle to keep pace with the complexity of modern financial transactions, especially with the rise of cryptocurrencies. Cryptocurrencies, due to their anonymity, have opened new avenues for illicit activities, making detection harder. To address this, AI and machine learning can analyze large datasets to detect hidden patterns of money laundering that traditional methods miss. Cryptography plays a key role by ensuring data confidentiality and integrity during analysis. The proposed system integrates AI, ML, and cryptographic techniques to enhance AML frameworks by predicting suspicious activities based on transaction behaviors. This approach improves detection accuracy while maintaining data privacy. Organizations like FATF set global standards to combat money laundering, but advanced technologies are essential to strengthen defenses as criminals develop more sophisticated laundering methods.

1.2 AIM OF THE PROJECT:

To develop an intelligent Anti-Money Laundering (AML) system that leverages artificial intelligence, machine learning, and cryptographic techniques to detect and prevent illicit financial activities within cryptocurrency transactions in real-time.

1.3 PROBLEM STATEMENT:

The rapid growth of cryptocurrency has opened new avenues for financial innovation but also created opportunities for money laundering due to its pseudonymous and decentralized nature. Traditional AML detection methods are inadequate for identifying complex laundering patterns within vast volumes of crypto transactions, leading to significant risks for financial institutions and regulators. Hence, there is a critical need for an advanced system capable of analyzing cryptocurrency transaction patterns intelligently to flag suspicious activities accurately and efficiently.

1.4 OBJECTIVE:

- To design and implement a machine learning-based AML framework that can:
    - Analyze large-scale cryptocurrency transactions.
    - Detect anomalies and suspicious transaction patterns in real-time.

- Integrate cryptographic techniques for secure data handling.
- Enhance the accuracy and speed of money laundering detection compared to traditional methods.
- Provide actionable insights to financial regulators and law enforcement agencies.

1.5 SCOPE:

 The system focuses on cryptocurrency-based financial transactions (such as Bitcoin, Ethereum).

 Applies AI and ML techniques including anomaly detection, clustering, and pattern recognition.

 Ensures data security and privacy through the use of advanced cryptography.

 Offers real-time monitoring and alerting for suspicious activities.

 Can be extended to multiple cryptocurrencies and adapted for integration into existing AML compliance frameworks.

 Supports scalability and continuous learning as new laundering techniques emerge.

1.6 INCLUSION AND EXCLUSION CRITERIA:

INCLUTION:

 Cryptocurrency transactions involving major coins (e.g., Bitcoin, Ethereum, Litecoin).

 Transactions recorded on public blockchain networks.

 Transaction datasets with attributes like wallet addresses, timestamps, transaction values, and types.

 Suspicious activity reports (SARs) and labeled datasets available for supervised machine learning.

 Patterns indicative of common laundering techniques (e.g., structuring, layering, mixing services).

EXCLUTION:

 Transactions in fiat (traditional currency) that are not linked to cryptocurrency exchanges.

 Private or permissioned blockchain transactions where data is not publicly accessible.

 Transaction datasets lacking sufficient features (e.g., missing timestamps, anonymized beyond traceability).

 Micropayments below a defined threshold unlikely to be involved in laundering (e.g., < $10).

1.6.1 SUMMARIZING SYSTEM:

- **Real-Time Detection:** Utilizes machine learning algorithms and real-time data processing to monitor cryptocurrency transactions continuously, detecting suspicious patterns and flagging potential money laundering activities as they occur.
- **Advanced Pattern Analysis:** Employs AI-driven techniques such as anomaly detection, clustering, and predictive modeling to analyze complex transaction behaviors across blockchain networks, uncovering hidden laundering methods.
- **Enhanced Security and Privacy:** Integrates cryptographic techniques to ensure the confidentiality, integrity, and authenticity of transaction data while maintaining compliance with regulatory standards.

1.6.2 ALGORITHM

Cryptography is the science of securing communication and information through the use of mathematical algorithms. Its primary goal is to protect sensitive data from unauthorized access, ensuring privacy, integrity, authentication, and non-repudiation in digital communications. Cryptography is essential in safeguarding online transactions, personal data, and even digital identities.

KEY CONCEPTS IN CRYPTOGRAPHY

1. **Encryption**
    - The process of converting plaintext (readable data) into ciphertext (unreadable data) to prevent unauthorized access. Only those with the correct decryption key can convert the ciphertext back into the original plaintext.
    - **Symmetric Encryption**: Uses the same key for both encryption and decryption. The key must be shared between the sender and receiver securely. Common algorithms include **AES (Advanced Encryption Standard)** and **DES (Data Encryption Standard)**.

- **Asymmetric Encryption**: Uses a pair of keys—one for encryption (public key) and one for decryption (private key). The public key is shared publicly, while the private key is kept secret. A common example is **RSA** (Rivest–Shamir–Adleman).

2. **Hashing**
   - Hashing transforms any input data (of any size) into a fixed-length string of characters, typically used for data integrity verification. Hashing is a one-way process, meaning it's nearly impossible to revert the hash back into its original data.
   - Common cryptographic hash functions include **SHA-256** (part of the SHA-2 family) and **MD5** (although MD5 is now considered broken and insecure).
3. **Digital Signatures**
   - A digital signature is a cryptographic method of verifying the authenticity of a digital message or document. It involves encrypting a hash of the message with a private key. The recipient can decrypt the hash with the sender's public key, verifying the integrity and authenticity of the message.
   - Digital signatures are widely used in various applications like email communication, software distribution, and cryptocurrency transactions.
4. **Public Key Infrastructure (PKI)**
   - PKI is a framework for managing digital keys and certificates. It ensures that communication between parties is encrypted and that the identity of users and systems is verified. It uses both public and private keys and relies on **Certificate Authorities (CAs)** to issue digital certificates that validate the legitimacy of public keys.
5. **Symmetric vs Asymmetric Cryptography**
   - **Symmetric Cryptography**: The same key is used for both encryption and decryption, making it faster and more efficient. The main challenge is the secure distribution of the key.
   - **Asymmetric Cryptography**: Uses two keys (public and private), solving the key distribution issue. It is slower than symmetric cryptography but allows for secure communication over an insecure channel, like the internet.

CHAPTER 2

LITERATURE SURVEY

2 LITERATURE SURVEY:

DETECTION OF MONEY LAUNDERING USING DATA MINING MODELS

A: REVIEW

Shazia Nawaz*1, Dr. Hamid Ghous*2

Money laundering (ML) is an illegal campaign to dissemble black money into clean money. Nowadays, the world banking system identification of money laundering has become a difficult task due to the large volume of banking transactions that eventually affect the global economy. ML fraud is a million-dollar business and rising every year. On the other side, Anti money laundering (AML) is the system that fights against money laundering. There are no useful AML methods which are available to prevent this unethical crime. However, financial action task force (FATF) is a global organization, which is working against financial crime like money laundering, credit card fraud. It was started in 1989 to fight against money laundering. Data mining methods have been widely used by many researchers to handle money laundering. This study contains detail review of various data mining methods like k-means clustering, hierarchical clustering, KNN, regression analysis, XGBoost and hybrid models used for detection and prediction of money laundering. We have also discussed the limitations of previous work and suggestion for the future research.

A MACHINE LEARNING APPROACH FOR ANTI-MONEY LAUNDERING

Shih, et al. (2019)

The paper explores the application of machine learning algorithms for detecting money laundering patterns in financial systems. The authors found that machine learning models outperformed traditional rule-based systems in detecting complex money laundering schemes.

CRYPTOCURRENCY AND ANTI-MONEY LAUNDERING: CHALLENGES AND SOLUTIONS

Zhang, et al. (2020)

This research addresses the challenges of using existing AML systems in the cryptocurrency domain. The authors propose using blockchain technology and cryptographic techniques to secure transaction data and enhance detection capabilities.

AI-POWERED MONEY LAUNDERING DETECTION IN FINANCIAL TRANSACTIONS

Liao, et al. (2021)

The paper examines how AI techniques, including neural networks and decision trees, can be used to predict suspicious financial transactions and prevent money laundering.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Traditional anti-money laundering systems rely on a set of predefined rules and manual monitoring processes to flag suspicious activities. These systems are often based on pattern recognition models that are predefined and require constant updates to remain effective. However, such rule-based systems are limited in their ability to adapt to new techniques used by money launderers. For example, these systems may struggle to detect sophisticated laundering methods such as layering, where illicit funds are hidden through complex transactions or cryptocurrencies.

Moreover, the existing systems are primarily designed for use in traditional banking systems, where transactions are centralized and easily tracked. With the rise of decentralized systems, especially cryptocurrency exchanges, current AML frameworks are ill-equipped to handle the anonymity and rapid volume of transactions involved in these markets. Existing solutions also rely heavily on human intervention, which can lead to delays in detecting fraudulent activities and result in missed opportunities to prevent illicit transactions.

3.1.1 Limitations of Existing Systems

1. Limited Adaptability: Traditional AML systems are often designed for fiat currency transactions and struggle to adapt to the unique, decentralized, and pseudonymous nature of cryptocurrency networks.
2. Static Rule-Based Approaches: Many systems rely heavily on static rule-based engines, which fail to detect novel or evolving laundering techniques that do not fit predefined patterns.
3. High False Positives: Current AI/ML-based models often generate a high rate of false positives, overwhelming compliance teams and reducing operational efficiency.
4. Lack of Comprehensive Data: Incomplete or anonymized blockchain data limits the effectiveness of machine learning models, making it difficult to trace the true source and destination of funds.
5. Latency in Detection: Some systems lack real-time monitoring capabilities, leading to delayed detection and slower response to suspicious activities.

6. Scalability Issues: Existing tools may face challenges in processing massive volumes of transactions, especially as crypto adoption grows globally.
7. Interoperability Limitations: Difficulty in integrating across different blockchain platforms and legacy systems hinders a unified view of laundering activities.
8. Privacy vs. Transparency Balance: Ensuring compliance while respecting user privacy remains a challenge, especially when handling encrypted or privacy-focused cryptocurrencies (like Monero).

3.2 PROPOSED SYSTEM

The proposed Anti-Money Laundering system combines the power of AI, machine learning, and cryptographic techniques to offer a more robust and efficient solution to detect and prevent money laundering. This system will utilize AI and ML algorithms to analyze transaction data and identify suspicious patterns, such as unusual amounts, frequent transactions, or complex transaction chains that are often characteristic of money laundering activities. These algorithms can learn from historical data and continuously improve their detection capabilities over time.

The cryptographic component of the system ensures that transaction data is encrypted and stored securely, maintaining the confidentiality of all involved parties. Blockchain technology can be integrated into the system to further enhance transparency, providing a secure and immutable record of all transactions. With the use of cryptography and AI, the proposed system can identify and flag illicit transactions in real-time, reducing the time taken to detect fraudulent activities and improving the overall accuracy of the AML process.

3.3 Advantages:

1. **Improved Detection Accuracy**: AI and ML algorithms can analyze complex patterns and detect anomalies that traditional systems might miss.
2. **Real-Time Detection**: The system can flag suspicious transactions in real-time, allowing for immediate intervention.
3. **Cryptographic Integrity**: The use of cryptography ensures the security, privacy, and integrity of transaction data, making it difficult for fraudsters to tamper with records.
4. **Adaptability**: Machine learning models can adapt to new fraud patterns, ensuring that the system remains effective even as money laundering techniques evolve.
5. **Scalability**: The system can scale to handle large volumes of transactions, making it suitable for global financial networks, including cryptocurrency markets.

6. **Cost-Effective**: By automating the detection process, the system reduces the need for manual intervention, saving time and resources.

3.4 DISADVANTAGES

1. **Ineffectiveness Against New Schemes**: Rule-based systems often fail to detect novel or complex money laundering techniques.
2. **Slow Detection Time**: Due to the volume of transactions, manual reviews and rule-based models lead to slower detection times.
3. **Limited Ability to Detect Cryptocurrency Laundering**: Traditional AML systems struggle with identifying suspicious cryptocurrency transactions due to the decentralized nature of cryptocurrencies.
4. **Lack of Adaptability**: Current systems are not adaptable to new fraud tactics or evolving money laundering strategies.
5. **High False Positive Rates**: Rule-based systems often flag legitimate transactions as suspicious, leading to inefficiency and wasted resources in manual investigations.

3.5 SYSTEM REQUIREMENTS

3.5.1 HARDWARE REQUIREMENTS

- **System** : Pentium i3 Processor
- **Hard Disk** : 500 GB
- **Monitor** : 15'' LED
- **Input Devices** : Keyboard, Mouse
- **Ram** : 4 GB

3.5.2 SOFTWARE REQUIREMENTS

- **Operating system** : Windows 10.
- **Coding Language** : Python 3.10
- **IDE** : Pycharm

CHAPTER-4

MODULE DESCRIPTION

4.1 DATA COLLECTION:

This step involves gathering transaction data from diverse financial sources, such as banks, cryptocurrency exchanges, and blockchain explorers. Comprehensive data acquisition ensures a rich dataset covering transaction histories, account details, and network flows for effective anti-money laundering analysis.

4.2 PREPORCESSING:

Preprocessing focuses on cleaning raw data to handle missing values, remove duplicates, and normalize inconsistent formats. It transforms the dataset into a structured, reliable format, ensuring that machine learning models receive high-quality input for accurate pattern recognition and anomaly detection.

4.3 FEATURE EXTRATION:

This step identifies and derives crucial features like transaction frequency, volume, sender-receiver relationships, and network topology metrics. These extracted features help the machine learning models understand behavioral patterns and enable deeper analysis of potentially suspicious financial activities.

4.4 MODEL TRAINING:

Machine learning models, including decision trees, random forests, and neural networks, are trained on labeled historical transaction data. The goal is to teach the models to recognize patterns and distinguish between legitimate and suspicious transactions with high accuracy.

4.5 CRYPTOGRAPHIC INTEGRITY CHECK:

Cryptographic methods like hashing and blockchain encryption are applied to maintain data security and integrity. This ensures transaction data is tamper-proof, verifiable, and immutable, providing trustworthiness and compliance with regulatory standards for sensitive financial information.

4.6 ANOMALY DETECTION:

The trained model analyzes new transaction data to spot deviations from normal patterns. Anomalies—such as unusually large transfers or complex transaction chains—are flagged in real time as potential indicators of money laundering, enabling quick response and investigation

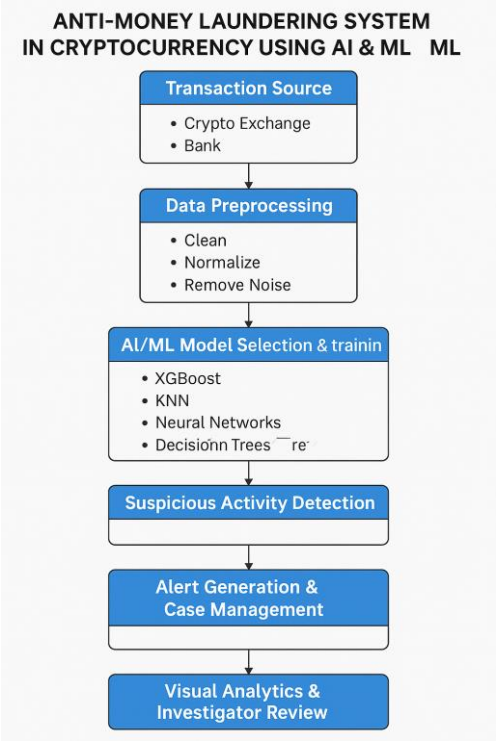
CHAPTER 5

SYSTEM DESIGN

5.1 PROCEDURAL DESIGN:

The relation upon the system is structure through a conceptual ER-Diagram, which not only Specifies the existential entities but also the standard relations through which the system exists And the personalities that are necessary for the system state to continue. The entity Relationship Diagram (ERD) depicts the relationship between the data objects. The set of primary components that are identified by the ERD are

1. Data object
2. Relationships
3. Attributes



5.2. DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through a system. It visualizes the processes, data stores. Data flows represent the movement of data between various components of the system. They are depicted as arrows and show the direction of data flow. Data flows are labelled to indicate the type of data being transmitted and external entities involved in a system and illustrates how data moves between them.

5.2.1 DFD SYMBOLS:

In the DFD, there are four symbols

1. A square defines a source(originator) or destination of system data.

2. An arrow identifies data flow. It is the pipeline through which the information flows

3. A circle or a bubble represents a process that transforms incoming data flow into outgoing data flows.

Diagram and Description:

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

5.2.2 DATA FLOW

The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have control flow and no loops or decision rules are present. Specific operations depending on the type of data can be explained by a flowchart. It is a graphical tool, useful for communicating with users, managers and other personnel. it is useful for analyzing existing as well as proposed system.

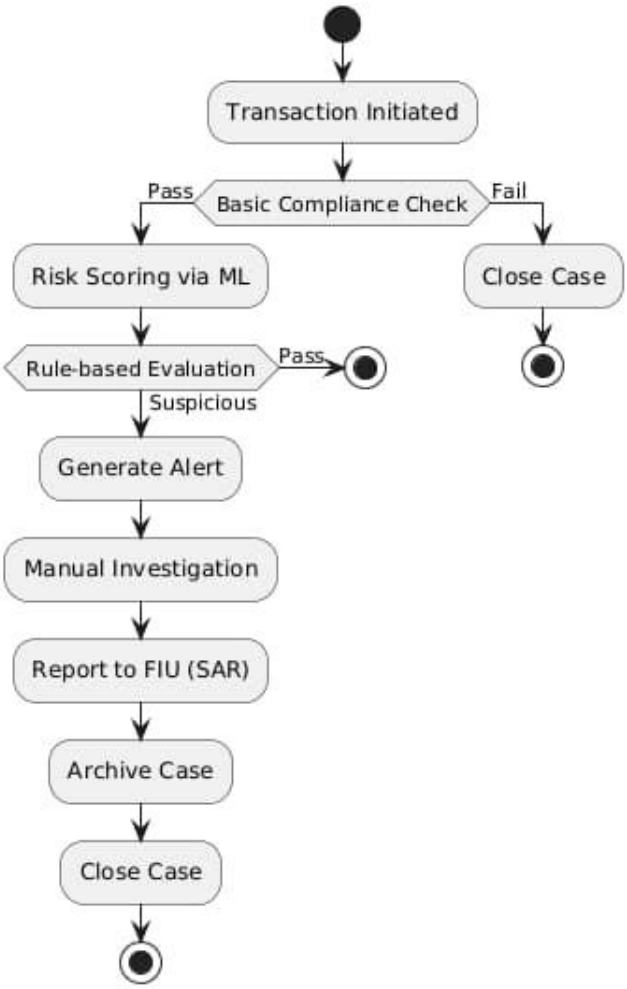
1. The processes are shown by named circles and data flows are represented

by named arrows entering or leaving the bubbles.

2. A rectangle represents a source or sink and it is a net originator or consumer of data. A source sink is typically outside the main system of study.

3. The processes should be numbered or put in ordered list to be referred easily.

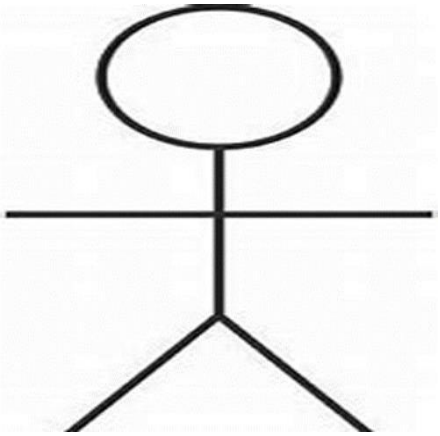
4. The arrow symbol is the symbol of data flow. A relatable name should be given to the flow to determine the information which is being moved.



5.3UML DIAGRAM

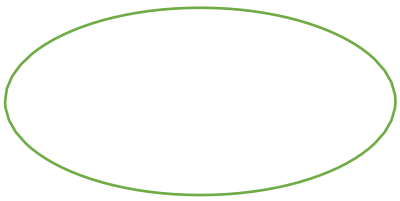
Actor:

Actors are typically individuals, roles, or other systems that interact with the system under consideration.



Use case:

A use case is a technique used in software engineering to capture and describe the interactions between a system and its users or other systems.



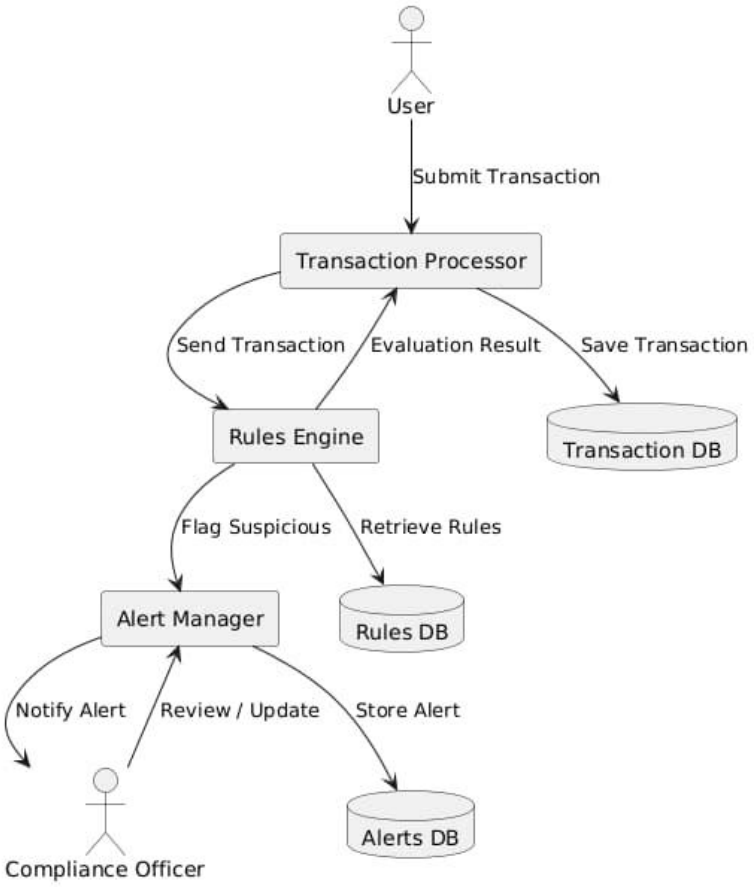
Unified Modelling Language (UML) is a standardized modelling language used in software engineering to visually represent and design software systems. It provides a set of graphical notations for expressing the architecture, structure, behaviour and interactions of complex systems.

There are various kinds of methods in software design: They are as follows,

- Use case Diagram
- Sequence Diagram
- Activity Diagram
- State chart Diagram

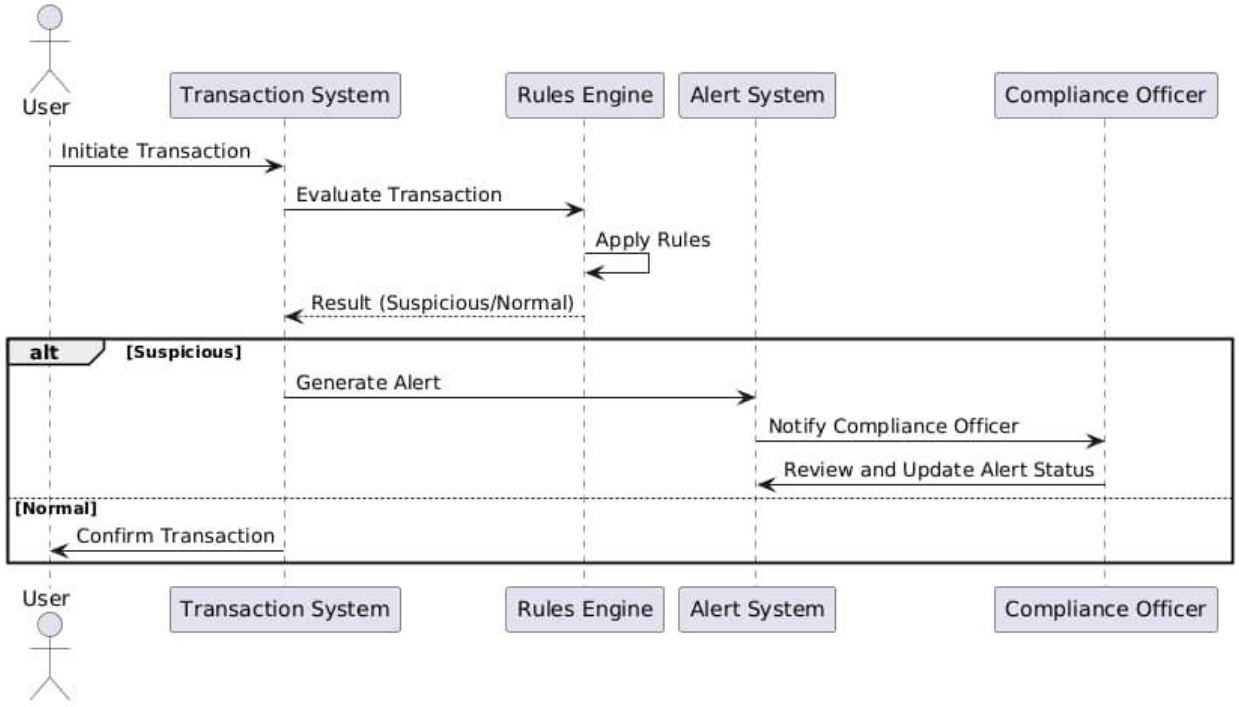
5.4 USE CASE DIAGRAMS:

A Use Case Diagram is a type of Unified Modelling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.



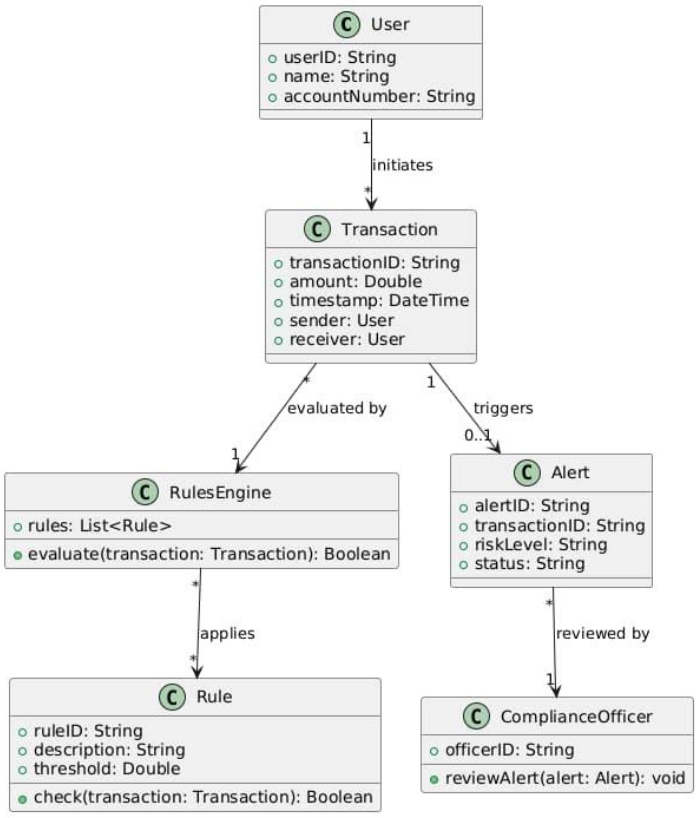
5.5 SEQUENCE DIAGRAM:

Sequence Diagram is an interaction diagram that details how operations are carried out -- what messages are sent and when. Sequence diagrams are organized according to time. The time progresses as you go down the page. The objects involved in the operation are listed from left to right according to when they take part in the message sequence.

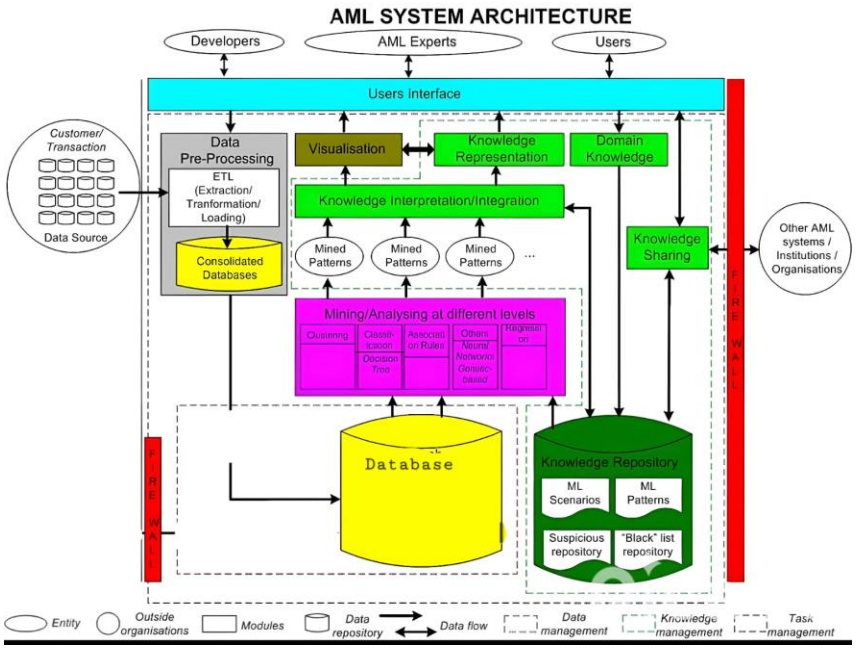


5.6 CLASS DIAGRAM:

Class diagrams are a type of UML (Unified Modelling Language) diagram used in software engineering to visually represent the structure and relationships of classes in a system. UML is a standardized modelling language that helps in designing and documenting software systems. They are an integral part of the software development process, helping in both the design and documentation phases.



5.7 ARCHITECTURE DESIGN



5.8 SYSTEM DESIGN

Software design sits at the technical kernel of the software engineering process and is applied regardless of the development paradigm and area of application. Design is the first step in the development phase for any engineered product or system. The designer's goal is to produce a model or representation of an entity that will later be built. Beginning, once system requirement have been specified and analyzed, system design is the first of the three technical activities - design, code and test that is required to build and verify software.

The importance can be stated with a single word "Quality". Design is the place where quality is fostered in software development. Design provides us with representations of software that can assess for quality. Design is the only way that we can accurately translate a customer's view into a finished software product or system. Software design serves as a foundation for all the software engineering steps that follow. Without a strong design we risk building an unstable system – one that will be difficult to test, one whose quality cannot be assessed until the last stage.

During design, progressive refinement of data structure, program structure, and procedural details are developed reviewed and documented. System design can be viewed from either technical or project management perspective. From the technical point of view, design is comprised of four activities – architectural design, data structure design, interface design and procedural design.

5.9 NORMALIZATION

It is a process of converting a relation to a standard form. The process is used to handle the problems that can arise due to data redundancy i.e. repetition of data in the database, maintain data integrity as well as handling problems that can arise due to insertion, updation, deletion anomalies.

Decomposing is the process of splitting relations into multiple relations to eliminate anomalies and maintain anomalies and maintain data integrity. To do this we use normal forms or rules for structuring relation.

- Insertion anomaly: Inability to add data to the database due to absence of other data.
- Deletion anomaly: Unintended loss of data due to deletion of other data.

- Update anomaly: Data inconsistency resulting from data redundancy and partial update
- Normal Forms: These are the rules for structuring relations that eliminate anomalies.

5.9.1 FIRST NORMAL FORM:

A relation is said to be in first normal form if the values in the relation are atomic for every attribute in the relation. By this we mean simply that no attribute value can be a set of values or, as it is sometimes expressed, a repeating group.

5.9.2 SECOND NORMAL FORM

A relation is said to be in second Normal form is it is in first normal form and it should satisfy any one of the following rules.

1) Primary key is a not a composite primary key
2) No non key attributes are present
3) Every non key attribute is fully functionally dependent on full set of primary key.

5.9.3 THIRD NORMAL FORM

A relation is said to be in third normal form if their exits no transitive dependencies.

Transitive Dependency

If two non key attributes depend on each other as well as on the primary key then they are said to be transitively dependent.

The above normalization principles were applied to decompose the data in multiple tables thereby making the data to be maintained in a consistent state.

CHAPTER 6

SYSTEM DEVELOPMENT

6.1 SOFTWARE DESCRIPTION

6.2 INTRODUCTION TO PYTHON:

Python is currently the most widely used multi-purpose, high-level programming language, which allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and the indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia
- Scientific computing
- Text processing and many more.

Python is currently the most widely used multi-purpose, high-level programming language, which allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and the indentation requirement of the language, makes them readable all the time.

Now that we know what makes Python a high level language, l et us also have a look at some of the lesser-known features of the language listed below:-

1. **Function annotations:** Python allows you to add annotations to function parameters and return types. These annotations are optional and do not affect the function's behaviour, but they can be used to provide additional information to developers working on the language.
2. **Coroutines:** Python supports coroutines, which are functions that can be paused and resumed. Coroutines are useful for writing asynchronous code, such as in web servers or networking applications.
3. **Enumerations:** Python has a built-in Enum class that allows you to define symbolic names for values. Enumerations are useful for improving the readability and maintainability of your code.
4. **List comprehensions:** Python allows you to create lists in a concise and readable way using list comprehensions. For example, you can create a list of squares of numbers using [x**2 for x in range(10)].
5. **Extended iterable unpacking:** Python allows you to unpack iterable (e.g., lists, tuples, and dictionaries) into variables using the * and ** operators. This feature makes it easy to work with complex data structures.
6. **The with the statement:** Python's with statement allows you to manage resources (such as files or network connections) cleanly and concisely. The with statement automatically opens and closes the resource, even in the case of exceptions.
7. **The walrus operator:** Python 3.8 introduced the walrus operator (:=), which allows you to assign a value to a variable as part of an expression. This feature is usedful for simplifying code and reducing the number of lines.
8. **The slots attribute:** Python allows you to optimize memory usage by using the slots attribute in classes. This attribute tells Python to allocate memory only for the specified attributes, reducing memory overhead.

Python is a popular programming language for multiple reasons, some of which include:

- **Simplicity:** Python is easy to learn and read, making it a number one choice for beginners. Its syntax is straightforward and consistent, allowing developers to write code quickly and efficiently.
- **Versatility:** Python is a general-purpose language, which means it can be used for a wide range of applications, including web development, data analysis, machine learning, and artificial intelligence.
- **Large community:** Python has a large and active community of developers who contribute to its development and offer support to new users. This community has created a vast array of libraries and frameworks that make development faster and easier.
- **Open-source:** Python is an open-source language, which means its source code is freely available and can be modified and distributed by anyone. This has led to the creation of many useful libraries and frameworks that have made Python even more popular.
- **Scalability:** Python can be used to build both small and large-scale applications. Its scalability makes it an attractive choice for startups and large organizations alike.

These are just a few of the lesser-known features of Python as a language. Python is a powerful and flexible language that offers many more features and capabilities that can help developers write efficient, maintainable, and scalable code with ease and efficiency.

6.3 LIBRARIES IN PYTHON

Normally, a library is a collection of books or is a room or place where many books are stored to be used later. Similarly, in the programming world, a library is a collection of precompiled codes that can be used later on in a program for some specific well-defined operations. Other than pre-compiled codes, a library may contain documentation, configuration data, message templates, classes, and values, etc.

A Python library is a collection of related modules. It contains bundles of code that can be used repeatedly in different programs. It makes Python Programming simpler and convenient for the programmer. As we don’t need to write the same code again and again for different programs.

Python libraries play a very vital role in fields of Machine Learning, Data Science, Data Visualization, etc.

6.4 WORKING OF PYTHON LIBRARY

As is stated above, a Python library is simply a collection of codes or modules of codes that we can use in a program for specific operations. We use libraries so that we don't need to write the code again in our program that is already available. But how it works. Actually, in the MS Windows environment, the library files have a DLL extension (Dynamic Load Libraries). When we link a library with our program and run that program, the linker automatically searches for that library. It extracts the functionalities of that library and interprets the program accordingly. That's how we use the methods of a library in our program. We will see further, how we bring in the libraries in our Python programs.

6.5 PYTHON STANDARD LIBRARY

The Python Standard Library contains the exact syntax, semantics, and tokens of Python. It contains built-in modules that provide access to basic system functionality like I/O and some other core modules. Most of the Python Libraries are written in the C programming language. The Python standard library consists of more than 200 core modules. All these work together to make Python a high-level programming language. Python Standard Library plays a very important role. Without it, the programmers can't have access to the functionalities of Python. But other than this, there are several other libraries in Python that make a programmer's life easier. Let's have a look at some of the commonly used libraries:

1. **TensorFlow:** This library was developed by Google in collaboration with the Brain Team. It is an open-source library used for high-level computations. It is also used in machine learning and deep learning algorithms. It contains a large number of tensor operations. Researchers also use this Python library to solve complex computations in Mathematics and Physics.

2. **Matplotlib:** This library is responsible for plotting numerical data. And that's why it is used in data analysis. It is also an open-source library and plots high-defined figures like pie charts, histograms, scatterplots, graphs, etc.
3. **Pandas:** Pandas are an important library for data scientists. It is an open-source machine learning library that provides flexible high-level data structures and a variety of analysis tools. It eases data analysis, data manipulation, and cleaning of data. Pandas support operations like Sorting, Re-indexing, Iteration, Concatenation, Conversion of data, Visualizations, Aggregations, etc.
4. **Numpy:** The name "Numpy" stands for "Numerical Python". It is the commonly used library. It is a popular machine learning library that supports large matrices and multi-dimensional data. It consists of in-built mathematical functions for easy computations. Even libraries like TensorFlow use Numpy internally to perform several operations on tensors. Array Interface is one of the key features of this library.
5. **SciPy:** The name "SciPy" stands for "Scientific Python". It is an open-source library used for high-level scientific computations. This library is built over an extension of Numpy. It works with Numpy to handle complex computations. While Numpy allows sorting and indexing of array data, the numerical data code is stored in SciPy. It is also widely used by application developers and engineers.
6. **Scrapy:** It is an open-source library that is used for extracting data from websites. It provides very fast web crawling and high-level screen scraping. It can also be used for data mining and automated testing of data.
7. **Scikit-learn:** It is a famous Python library to work with complex data. Scikit-learn is an open-source library that supports machine learning. It supports variously supervised and unsupervised algorithms like linear regression, classification, clustering, etc. This library works in association with Numpy and SciPy.
8. **PyGame:** This library provides an easy interface to the Standard Directmedia Library (SDL) platform-independent graphics, audio, and input libraries. It is used for developing video games using computer graphics and audio libraries along with Python programming language.

9. **PyTorch:**PyTorch is the largest machine learning library that optimizes tensor computations. It has rich APIs to perform tensor computations with strong GPU acceleration. It also helps to solve application issues related to neural networks.
10. **PyBrain:** The name "PyBrain" stands for Python Based Reinforcement Learning, Artificial Intelligence, and Neural Networks library. It is an open-source library built for beginners in the field of Machine Learning. It provides fast and easy-to-use algorithms for machine learning tasks. It is so flexible and easily understandable and that's why is really helpful for developers that are new in research fields.

There are many more libraries in Python. We can use a suitable library for our purposes.

6.6 USE OF LIBRARIES IN PYTHON

As we write large-size programs in Python, we want to maintain the code's modularity. For the easy maintenance of the code, we split the code into different parts and we can use that code later ever we need it. In Python, modules play that part. Instead of using the same code in different programs and making the code complex, we define mostly used functions in modules and we can just simply import them in a program wherever there is a requirement. We don't need to write that code but still, we can use its functionality by importing its module. Multiple interrelated modules are stored in a library. And whenever we need to use a module, we import it from its library. In Python, it's a very simple job to do due to its easy syntax. We just need to use import.

Guido van Rossum began working on Python in the late 1980s as a successor to the ABC programming language and first released it in 1991 as Python 0.9.0. Python 2.0 was released in 2000. Python 3.0, released in 2008, was a major revision not completely backward-compatible with earlier versions. Python 2.7.18, released in 2020, was the last release of Python 2.

Python consistently ranks as one of the most popular programming languages.

Python was conceived in the late 1980s by Guido van Rossum at Centrum Wiskunde&Informatica (CWI) in the Netherlands as a successor to the ABC programming language, which was inspired by SETL, capable of exception handling and interfacing with the Amoeba operating system. Its implementation began in December 1989. Van Rossum shouldered sole responsibility for the project, as the lead developer, until 12 July 2018, when he

announced his "permanent vacation" from his responsibilities as Python's "benevolent dictator for life", a title the Python community bestowed upon him to reflect his long-term commitment as the project's chief decision-maker. In January 2019, active Python core developers elected a five-member Steering Council to lead the project.

Python 2.0 was released on 16 October 2000, with many major new features such as list comprehensions, cycledetecting garbagecollection, referencecounting,and Unicode support. Pyth on 3.0, released on 3 December 2008, with many of its major features backported to Python 2.6.x and 2.7.x. Releases of Python 3 include the 2to3 utility, which automates the translation of Python 2 code to Python 3.

Python 2.7's end-of-life was initially set for 2015, then postponed to 2020 out of concern that a large body of existing code could not easily be forward-ported to Python 3. No further security patches or other improvements will be released for it. Currently only 3.7 and later are supported. In 2021, Python 3.9.2 and 3.8.8 were expedited as all versions of Python (including 2.7) had security issues leading to possible remote code execution and web cache poisoning.

In 2022, Python 3.10.4 and 3.9.12 were expedited and 3.8.13, and 3.7.13, because of many security issues. When Python 3.9.13 was released in May 2022, it was announced that the 3.9 series (joining the older series 3.8 and 3.7) would only receive security fixes in the future. On September 7, 2022, four new releases were made due to a potential denial-of-service attack: 3.10.7, 3.9.14, 3.8.14, and 3.7.14. As of November 2022, Python 3.11.0 is the current stable release. Notable changes from 3.10 include increased program execution speed and improved error reporting.

6.7 DESIGN PHILOSOPHY AND FEATURES

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are fully supported, and many of their features support functional programming and aspect-oriented programming (including metaprogramming and metaobjects). Many other paradigms are supported via extensions, including design by contract and logic programming.

Python uses dynamic typing and a combination of reference counting and a cycle-detecting garbage collector for memory management. It uses dynamic name resolution (late binding), which binds method and variable names during program execution.

Its design offers some support for functional programming in the Lisp tradition. The standard library has two modules (itertools and functools) that implement functional tools borrowed from Haskell and Standard ML.

Its core philosophy is summarized in the document The Zen of Python (PEP 20), which includes aphorisms such as:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Readability counts.

Rather than building all of its functionality into its core, Python was designed to be highly extensible via modules. This compact modularity has made it particularly popular as a means of adding programmable interfaces to existing applications. Van Rossum's vision of a small core language with a large standard library and easily extensible interpreter stemmed from his frustrations with ABC, which espoused the opposite approach.

Python strives for a simpler, less-cluttered syntax and grammar while giving developers a choice in their coding methodology. In contrast to Perl's "there is more than one way to do it" motto, Python embraces a "there should be one—and preferably only one—obvious way to do it" philosophy. Alex Martelli, a Fellow at the Python Software Foundation and Python book author, wrote: "To describe something as 'clever' is not considered a compliment in the Python culture."

Python's developers strive to avoid premature optimization and reject patches to non-critical parts of the CPython reference implementation that would offer marginal increases in speed at the cost of clarity. When speed is important, a Python programmer can move time-critical functions to extension modules written in languages such as C; or use PyPy, a just-in-time

compiler. Cython is also available, which translates a Python script into C and makes direct C-level API calls into the Python interpreter.

Python's developers aim for it to be fun to use. This is reflected in its name—a tribute to the British comedy group Monty Pythonand in occasionally playful approaches to tutorials and reference materials, such as the use of the terms "spam" and "eggs" (a reference to a Monty Python sketch) in examples, instead of the often-used "foo" and "bar".

A common neologism in the Python community is pythonic, which has a wide range of meanings related to program style. "Pythonic" code may use Python idioms well, be natural or show fluency in the language, or conform with Python's minimalist philosophy and emphasis on readability. Code that is difficult to understand or reads like a rough transcription from another programming language is called unpythonic.

6.8 SYNTAX AND SEMANTICS

Python is meant to be an easily readable language. Its formatting is visually uncluttered and often uses English keywords where other languages use punctuation. Unlike many other languages, it does not use curly brackets to delimit blocks, and semicolons after statements are allowed but rarely used. It has fewer syntactic exceptions and special cases than C or Pascal.

Python uses whitespace indentation, rather than curly brackets or keywords, to delimit blocks. An increase in indentation comes after certain statements; a decrease in indentation signifies the end of the current block. Thus, the program's visual structure accurately represents its semantic structure. This feature is sometimes termed the off-side rule. Some other languages use indentation this way; but in most, indentation has no semantic meaning. The recommended indent size is four spaces.

Statements and control flow

Python's statements include:

- The assignment statement, using a single equals sign =
- The if statement, which conditionally executes a block of code, along with else and elif (a contraction of else-if)

- The for statement, which iterates over an iterable object, capturing each element to a local variable for use by the attached block
- The while statement, which executes a block of code as long as its condition is true
- The try statement, which allows exceptions raised in its attached code block to be caught and handled by except clauses (or new syntax except* in Python 3.11 for exception groups[85]); it also ensures that clean-up code in a finally block is always run regardless of how the block exits
- The raise statement, used to raise a specified exception or re-raise a caught exception
- The class statement, which executes a block of code and attaches its local namespace to a class, for use in object-oriented programming
- The def statement, which defines a function or method
- The with statement, which encloses a code block within a context manager (for example, acquiring a lock before it is run, then releasing the lock; or opening and closing a file), allowing resource-acquisition-is-initialization (RAII)-like behavior and replacing a common try/finally idiom
- The break statement, which exits a loop
- The continue statement, which skips the rest of the current iteration and continues with the next
- The del statement, which removes a variable—deleting the reference from the name to the value, and producing an error if the variable is referred to before it is redefined
- The pass statement, serving as a NOP, syntactically needed to create an empty code block
- The assert statement, used in debugging to check for conditions that should apply
- The yield statement, which returns a value from a generator function (and also an operator); used to implement coroutines
- The return statement, used to return a value from a function
- The import and from statements, used to import modules whose functions or variables can be used in the current program

The assignment statement (=) binds a name as a reference to a separate, dynamically allocated object. Variables may subsequently be rebound at any time to any object. In Python, a variable name is a generic reference holder without a fixed data type; however, it

always refers to some object with a type. This is called dynamic typing—in contrast to statically-typed languages, where each variable may contain only a value of a certain type.

Python does not support tail call optimization or first-class continuations, and, according to Van Rossum, it never will. However, better support for coroutine-like functionality is provided by extending Python's generators. Before 2.5, generators were lazy iterators; data was passed unidirectionally out of the generator. From Python 2.5 on, it is possible to pass data back into a generator function; and from version 3.3, it can be passed through multiple stack levels.

6.9 DEVELOPMENT

Python's development is conducted largely through the Python Enhancement Proposal (PEP) process, the primary mechanism for proposing major new features, collecting community input on issues, and documenting Python design decisions. Python coding style is covered in PEP 8 Outstanding PEPs are reviewed and commented on by the Python community and the steering council.

Enhancement of the language corresponds with the development of the CPython reference implementation. The mailing list python-dev is the primary forum for the language's development. Specific issues were originally discussed in the Roundup bug tracker hosted at by the foundation. In 2022, all issues and discussions were migrated to GitHub. Development originally took place on a self-hosted source-code repository running Mercurial, until Python moved to GitHub in January 2017.

CPython's public releases come in three types, distinguished by which part of the version number is incremented:

- Backward-incompatible versions, where code is expected to break and needs to be manually ported. The first part of the version number is incremented. These releases happen infrequently—version 3.0 was released 8 years after 2.0. According to Guido van Rossum, a version 4.0 is very unlikely to ever happen. Major or "feature" releases are largely compatible with the previous version but introduce new features. The second part of the version number is incremented. Starting with Python 3.9, these releases are expected to happen annually. Each major version is supported by bug fixes for several years after its release.

- Bugfix releases, which introduce no new features, occur about every 3 months and are made when a sufficient number of bugs have been fixed upstream since the last release. Security vulnerabilities are also patched in these releases. The third and final part of the version number is incremented.

Many alpha, beta, and release-candidates are also released as previews and for testing before final releases. Although there is a rough schedule for each release, they are often delayed if the code is not ready. Python's development team monitors the state of the code by running the large unit test suite during development.

The major academic conference on Python is PyCon. There are also special Python mentoring programs, such as Pyladies.

Python 3.10 deprecated wstr (to be removed in Python 3.12; meaning Python extensions need to be modified by then), and added pattern matching to the language.

6.9.1 API documentation generators

Tools that can generate documentation for Python API include pydoc (available as part of the standard library), Sphinx, Pdoc and its forks, Doxygen and Graphviz, among others.

6.9.2 Naming

Python's name is derived from the British comedy group Monty Python, whom Python creator Guido van Rossum enjoyed while developing the language. Monty Python references appear frequently in Python code and culture; for example, the metasyntactic variables often used in Python literature are spam and eggs instead of the traditional foo and bar. The official Python documentation also contains various references to Monty Python routines. The prefix Py- is used to show that something is related to Python. Examples of the use of this prefix in names of Python applications or libraries include Pygame, a binding of SDL to Python (commonly used to create games); PyQt and PyGTK, which bind Qt and GTK to Python respectively; and PyPy, a Python implementation originally written in Python.

PopularitSince 2003, Python has consistently ranked in the top ten most popular programming languages in the TIOBE Programming Community Index where as of December 2022 it was the most popular language (ahead of C, C++, and Java). It was selected

Programming Language of the Year (for "the highest rise in ratings in a year") in 2007, 2010, 2018, and 2020 (the only language to have done so four times as of 202).An empirical study found that scripting languages, such as Python, are more productive than conventional languages, such as C and Java, for programming problems involving string manipulation and search in a dictionary.

6.10 OPEN CV

OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. Being an Apache 2 licensed product, OpenCV makes it easy for businesses to utilize and modify the code.

The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects, produce 3D point clouds from stereo cameras, stitch images together to produce a high resolution image of an entire scene, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality, etc. OpenCV has more than 47 thousand people of user community and estimated number of downloads exceeding 18 million. The library is used extensively in companies, research groups and by governmental bodies.

Along with well-established companies like Google, Yahoo, Microsoft, Intel, IBM, Sony, Honda, Toyota that employ the library, there are many startups such as Applied Minds, VideoSurf, and Zeitera, that make extensive use of OpenCV. OpenCV’s deployed uses span the range from stitching streetview images together, detecting intrusions in surveillance video in Israel, monitoring mine equipment in China, helping robots navigate and pick up objects at Willow Garage, detection of swimming pool drowning accidents in Europe, running interactive art in Spain and New York, checking runways for debris in Turkey, inspecting labels on products in factories around the world on to rapid face detection in Japan.

It has C++, Python, Java and MATLAB interfaces and supports Windows, Linux, Android and Mac OS. OpenCV leans mostly towards real-time vision applications and takes advantage of MMX and SSE instructions when available. A full-featured CUDAandOpenCL interfaces are being actively developed right now. There are over 500 algorithms and about 10 times as many functions that compose or support those algorithms. OpenCV is written natively in C++ and has a templated interface that works seamlessly with STL containers.

6.11 OPENCV LIBRARY MODULES

Following are the main library modules of the OpenCV library.

6.11.1 Core Functionality

This module covers the basic data structures such as Scalar, Point, Range, etc., that are used to build OpenCV applications. In addition to these, it also includes the multidimensional array Mat, which is used to store the images. In the Java library of OpenCV, this module is included as a package with the name org.opencv.core.

6.11.2 Image Processing

This module covers various image processing operations such as image filtering, geometrical image transformations, color space conversion, histograms, etc. In the Java library of OpenCV, this module is included as a package with the name org.opencv.imgproc.

6.11.3 Video

This module covers the video analysis concepts such as motion estimation, background subtraction, and object tracking. In the Java library of OpenCV, this module is included as a package with the name org.opencv.video.

6.11.4 Video I/O

This module explains the video capturing and video codecs using OpenCV library. In the Java library of OpenCV, this module is included as a package with the name org.opencv.videoio.

6.11.5 calib3d

This module includes algorithms regarding basic multiple-view geometry algorithms, single and stereo camera calibration, object pose estimation, stereo correspondence and elements of3D reconstruction. In the Java library of OpenCV, this module is included as a package with the name org.opencv.calib3d.

6.11.6 Features2d

This module includes the concepts of feature detection and description. In the Java library of OpenCV, this module is included as a package with the name org.opencv.features2d.

6.11.7 Objdetect

This module includes the detection of objects and instances of the predefined classes such as faces, eyes, mugs, people, cars, etc. In the Java library of OpenCV, this module is included as a package with the name org.opencv.objdetect.

6.11.8 Highgui

This is an easy-to-use interface with simple UI capabilities. In the Java library of OpenCV, the features of this module is included in two different packages namely, org.opencv.imgcodecs and org.opencv.videoio.

6.12 PILLOW

Digital Image processing means processing the image digitally with the help of a computer. Using image processing we can perform operations like enhancing the image, blurring the image, extracting text from images, and many more operations. There are various ways to process images digitally. Here we will discuss the Pillow module of Python.

Python Pillow is built on the top of PIL (Python Image Library) and is considered as the fork for the same as PIL has been discontinued from 2011.

Pillow supports many image file formats including BMP, PNG, JPEG, and TIFF. The library encourages adding support for newer formats in the library by creating new file decoders.

The Pillow module provides the open() and show() function to read and display the image respectively. For displaying the image Pillow first converts the image to a .png format (on Windows OS) and stores it in a temporary buffer and then displays it. Therefore, due to the conversion of the image format to .png some properties of the original image file format might be lost (like animation). Therefore, it is advised to use this method only for test purposes.

6.13 TENSORFLOW

TensorFlow is an end-to-end open source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries, and community resources that lets researchers push the state-of-the-art in ML, and gives developers the ability to easily build and deploy ML-powered applications.

TensorFlow provides a collection of workflows with intuitive, high-level APIs for both beginners and experts to create machine learning models in numerous languages. Developers have the option to deploy models on a number of platforms such as on servers, in the cloud, on mobile and edge devices, in browsers, and on many other JavaScript platforms. This enables developers to go from model building and training to deployment much more easily.

6.13.1 Easy model building

Build and train ML models easily using intuitive high-level APIs like Keras with eager execution, which makes for immediate model iteration and easy debugging.

6.13.2 Robust ML production anywhere

Easily train and deploy models in the cloud, on-prem, in the browser, or on-device no matter what language you use.

6.13.3 Powerful experimentation for research

A simple and flexible architecture to take new ideas from concept to code, to state-of-the-art models, and to publication faster.

- **TensorFlow** was originally developed by researchers and engineers working on the Google Brain Team within Google's Machine Intelligence research organization for the

purposes of conducting machine learning and deep neural networks research, but the system is general enough to be applicable in a wide variety of other domains as well! Let us first try to understand what the word **TensorFlow** actually mean! **TensorFlow** is basically a software library for numerical computation using **data flow graphs** where:

- **Nodes** in the graph represent mathematical operations.
- **Edges** in the graph represent the multidimensional data arrays (called **tensors**) communicated between them. (Please note that **tensor** is the central unit of data in TensorFlow).
-

6.14 TORCH

Deep Learning is a branch of Machine Learning where algorithms are written which mimic the functioning of a human brain. The most commonly used libraries in deep learning are Tensorflow and PyTorch. As there are various deep learning frameworks available, one might wonder when to use PyTorch. Here are reasons why one might prefer using Pytorch for specific tasks.

Pytorch is an open-source deep learning framework available with a Python and C++ interface. Pytorch resides inside the torch module. In PyTorch, the data that has to be processed is input in the form of a tensor.

CHAPTER 7

SYSTEM TESTING

Software testing is a critical element of software quality assurance and represents the ultimate review of software specification, design and coding. The increasing visibility of software as a system element and the attendant "costs" associated with a software failure are motivating forces for conference management system project well planned, thorough testing. It is not unusual for conference management system project software. Development organization to expend 40 percent of total project effort on testing. Hence the importance of software testing and its implications with respect to software quality can not be overemphasized. Different types of testing have been carried out for conference management system project this system, and they are briefly explained below.

7.1 TYPES OF TESTS

Testing is the process of trying to discover every conceivable fault or weakness in a work product. The different types of testing are given below:

7.1.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration.

This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

7.1.2 INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

7.1.3 VALIDATION TESTING

Tests were performed to find conformity with the requirements. Plans and procedures were designed to ensure that all functional requirements are satisfied. The software was alpha-tested there are two goals in preparing test plans. Firstly, a properly detailed test plan demonstrates that the program specifications are understood completely. Secondly, the test plan is used during program testing to prove the correctness of the program.

CHAPTER 8

SOURCE CODES

```
from flask import Flask, render_template, request
import pandas as pd
from sklearn.ensemble import IsolationForest
import networkx as nx
import matplotlib.pyplot as plt
import os

app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = 'uploads'
app.config['STATIC_FOLDER'] = 'static'

# Global storage for processed data
report_data = {}

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/upload', methods=['GET', 'POST'])
def upload():
    global report_data

    if request.method == 'POST':
        file = request.files['file']

        if file and file.filename.endswith('.csv'):
            df = pd.read_csv(file)
            save_path = os.path.join(app.config['UPLOAD_FOLDER'], 'latest.csv')
```

```
        df.to_csv(save_path, index=False)

        if 'timestamp' in df.columns:
            df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')

        if 'amount' in df.columns and 'sender' in df.columns and 'receiver' in df.columns:
            model = IsolationForest(contamination=0.1)
            df['risk_score'] = model.fit_predict(df[['amount']])

            suspicious_receivers = df[df['risk_score'] == -1]['receiver'].unique().tolist()
            suspicious_senders =
df[df['receiver'].isin(suspicious_receivers)]['sender'].unique().tolist()

            receiver_features = (
                df.groupby('receiver')
                .agg(
                    unique_senders=('sender', 'nunique'),
                    transaction_count=('amount', 'count'),
                    total_amount=('amount', 'sum'),
                    avg_amount=('amount', 'mean'),
                    std_amount=('amount', 'std'),
                    time_span=('timestamp', lambda x: (x.max() - x.min()).total_seconds())
                )
                .reset_index()
            )

            sender_frequency = df[df['receiver'].isin(suspicious_receivers)][
                'sender'].value_counts().to_frame().reset_index()
            sender_frequency.columns = ['sender', 'count']

            suspicious_senders_df = df[df['sender'].isin(suspicious_senders)]
```

```
    controller_senders = suspicious_senders_df.groupby('sender').filter(
        lambda x: len(x['receiver'].unique()) > 1)['sender'].unique().tolist()

    controller_transactions = df[
        (df['sender'].isin(controller_senders)) &
        (df['receiver'].isin(suspicious_receivers))
        ][
        ['transaction_id', 'sender', 'receiver', 'amount', 'timestamp']
    ]

    # Smurfing pattern detection: acc1 → [acc2, acc3] → accE
    smurfing_paths = []
    for sender in df['sender'].unique():
        level1 = df[df['sender'] == sender]['receiver'].unique()
        for intermediate in level1:
            level2_df = df[df['sender'] == intermediate]
            for final_receiver in level2_df['receiver'].unique():
                smurfing_paths.append((sender, intermediate, final_receiver))

    smurf_df = pd.DataFrame(smurfing_paths, columns=['root_sender', 'middle_sender',
'final_receiver'])
    smurf_df = smurf_df[smurf_df['root_sender'] != smurf_df['middle_sender']]

    # Dashboard summary
    total_transactions = len(df)
    total_amount = df['amount'].sum()
    most_active_sender = df['sender'].value_counts().idxmax()
    most_active_receiver = df['receiver'].value_counts().idxmax()

    report_data = {
        'receiver_features': receiver_features.to_html(classes='table table-bordered',
```

```
index=False),
            'suspicious_receivers': suspicious_receivers,
            'suspicious_senders': suspicious_senders,
            'sender_frequency_html': sender_frequency.to_html(classes='table table-bordered',
index=False),
            'controller_transactions_html': controller_transactions.to_html(classes='table table-
bordered', index=False),
            'smurf_df_html': smurf_df.to_html(classes='table table-bordered', index=False),
            'total_transactions': total_transactions,
            'total_amount': total_amount,
            'most_active_sender': most_active_sender,
            'most_active_receiver': most_active_receiver,
            'recent_transactions_html': df.tail(10).to_html(classes='table table-striped',
index=False)
        }

        # Generate and save network graph
        G = nx.DiGraph()
        for _, row in df.iterrows():
            sender = row['sender']
            receiver = row['receiver']
            risk = ' (SUSP)' if receiver in suspicious_receivers else ''
            G.add_edge(sender, receiver + risk, weight=row['amount'])

        plt.figure(figsize=(12, 8))
        pos = nx.spring_layout(G, k=0.5)
        nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=2000,
edge_color='gray', font_size=10)
        edge_labels = nx.get_edge_attributes(G, 'weight')
        nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
```

```
            graph_path = os.path.join(app.config['STATIC_FOLDER'], 'graph.png')
            plt.savefig(graph_path)
            plt.close()

            return render_template('upload.html', message='File processed successfully.')

    return render_template('upload.html', message='')


@app.route('/fetch')
def fetch():
    file_path = os.path.join(app.config['UPLOAD_FOLDER'], 'latest.csv')
    if not os.path.exists(file_path):
        return render_template('fetch.html', message='No file uploaded yet.')

    df = pd.read_csv(file_path)
    all_transactions_html = df.to_html(classes='table table-bordered', index=False)

    return render_template('fetch.html', all_transactions=all_transactions_html)

@app.route('/report')
def report():
    if not report_data:
        return render_template('report.html', message='No data processed yet.')
    return render_template('report.html', **report_data)

@app.route('/visualize')
def visualize():
    return render_template('visualize.html', graph_path='static/graph.png')

@app.route('/dashboard')
```

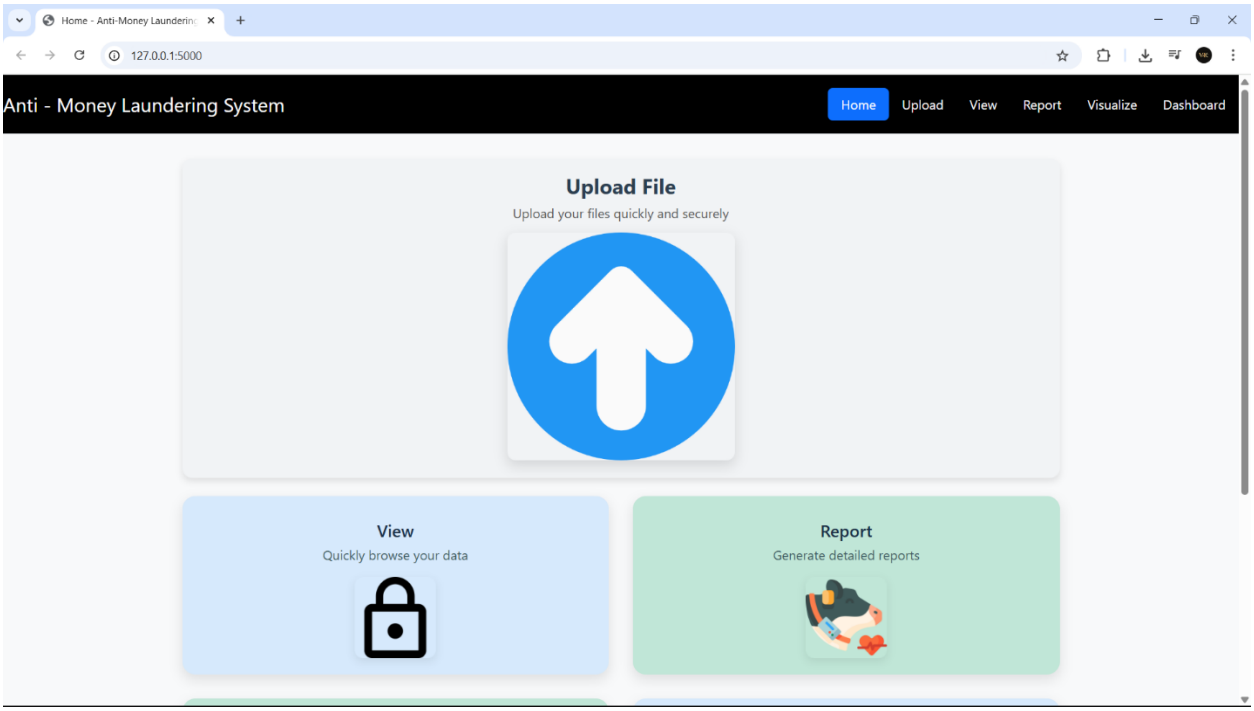
```
def dashboard():
    if not report_data:
        return render_template('dashboard.html', message='No data available. Please upload a file first.')
    return render_template('dashboard.html', **report_data)

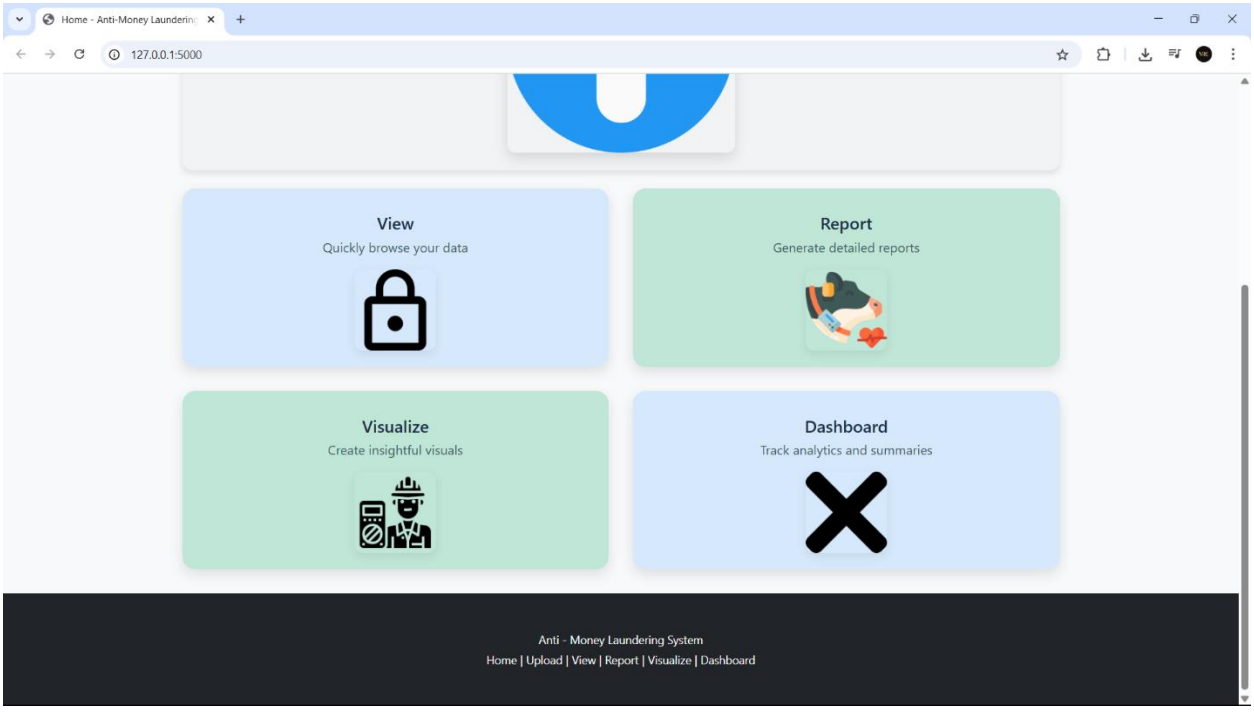
if __name__ == '__main__':
    os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
    os.makedirs(app.config['STATIC_FOLDER'], exist_ok=True)
    app.run(debug=True)
```

CHAPTER 9

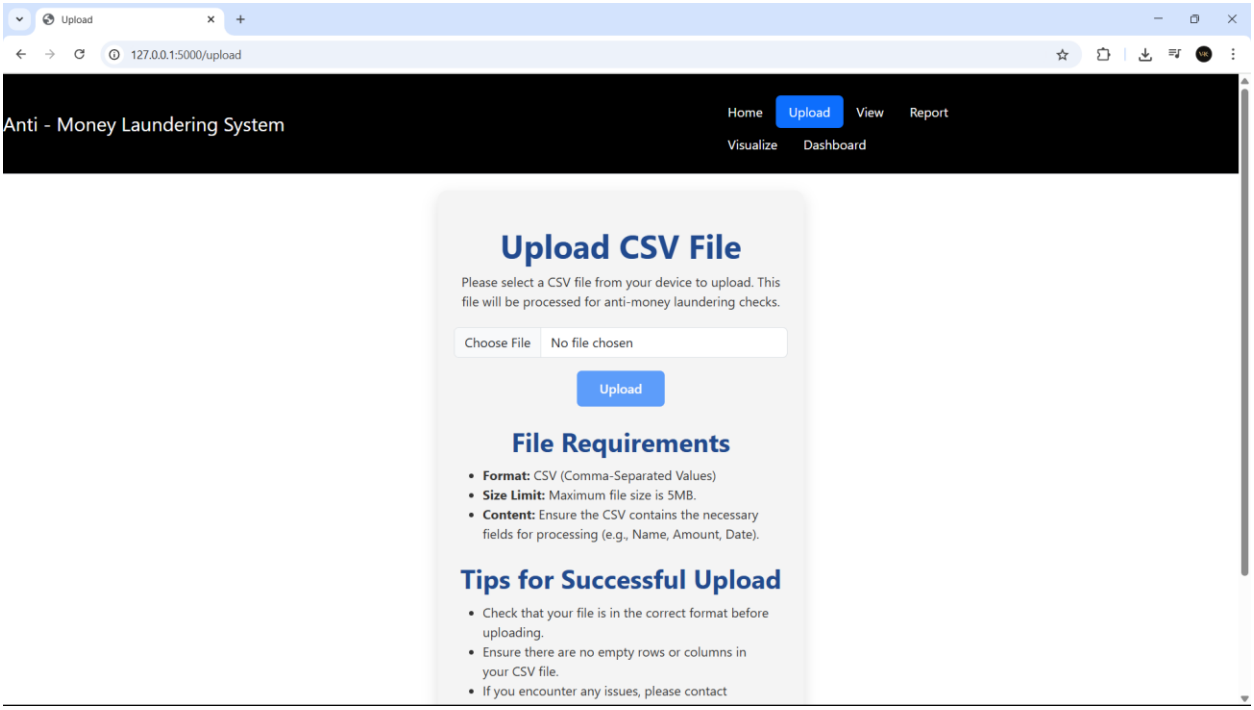
OUTPUT

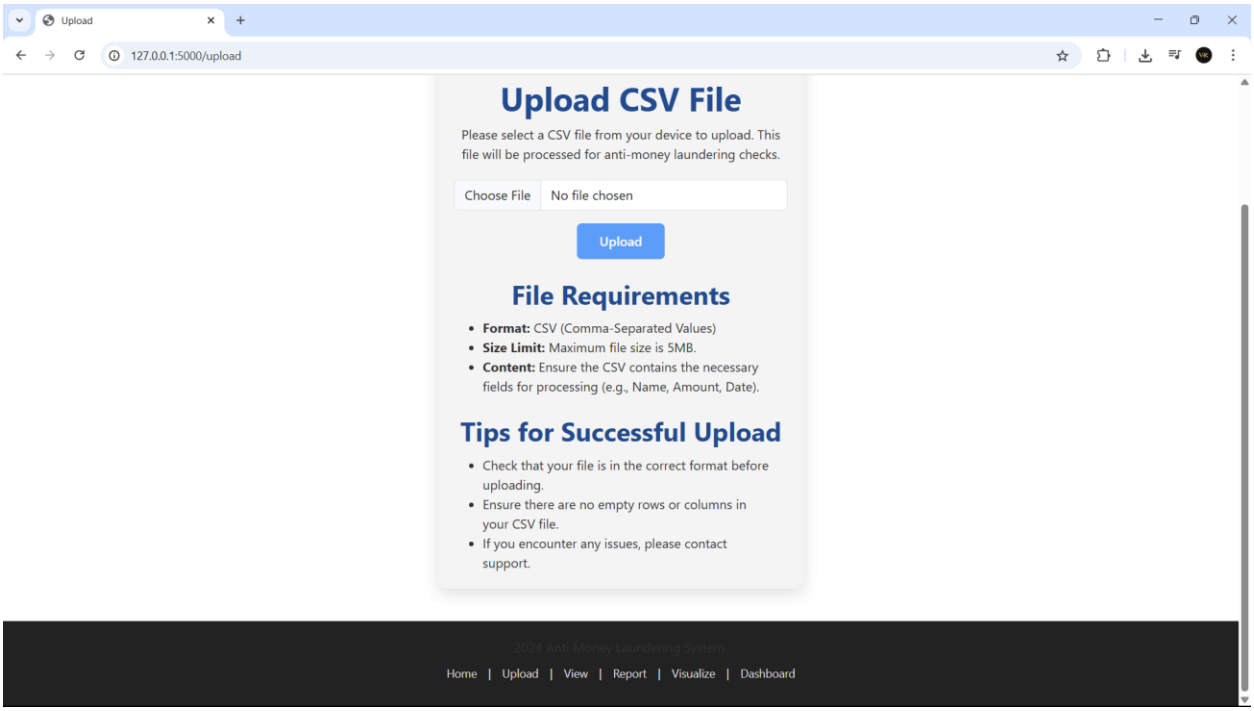
Home page



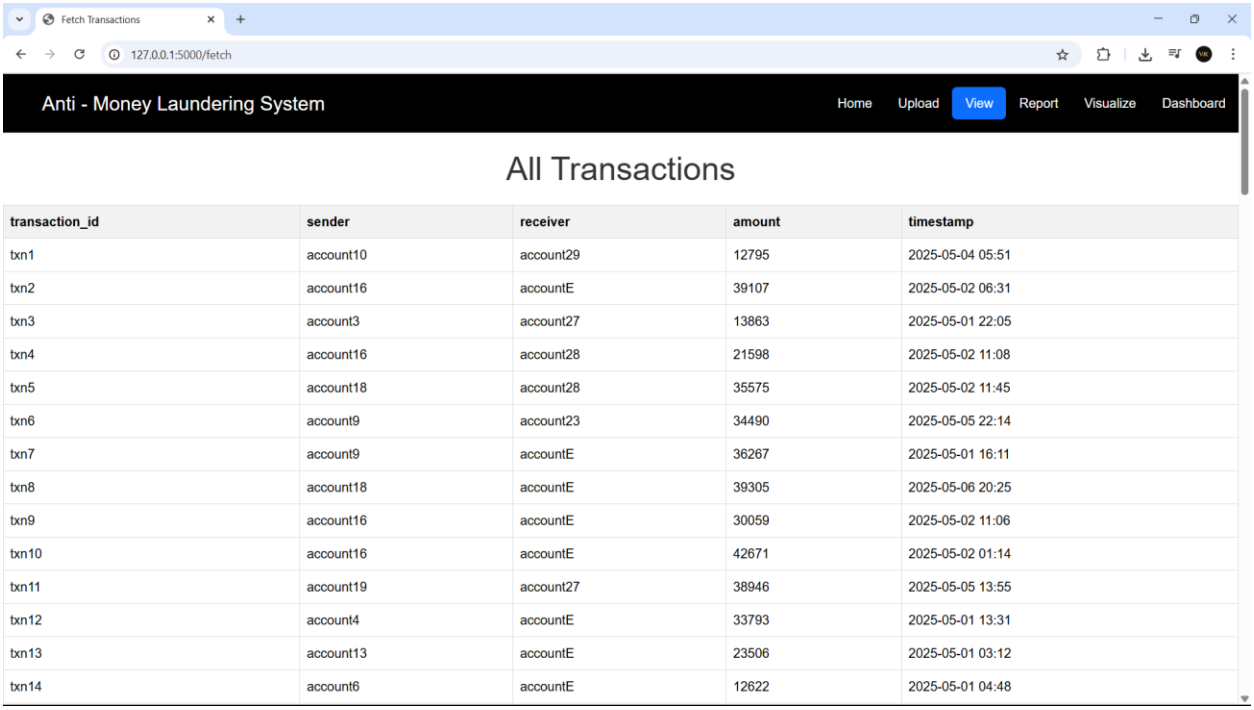


Upload page

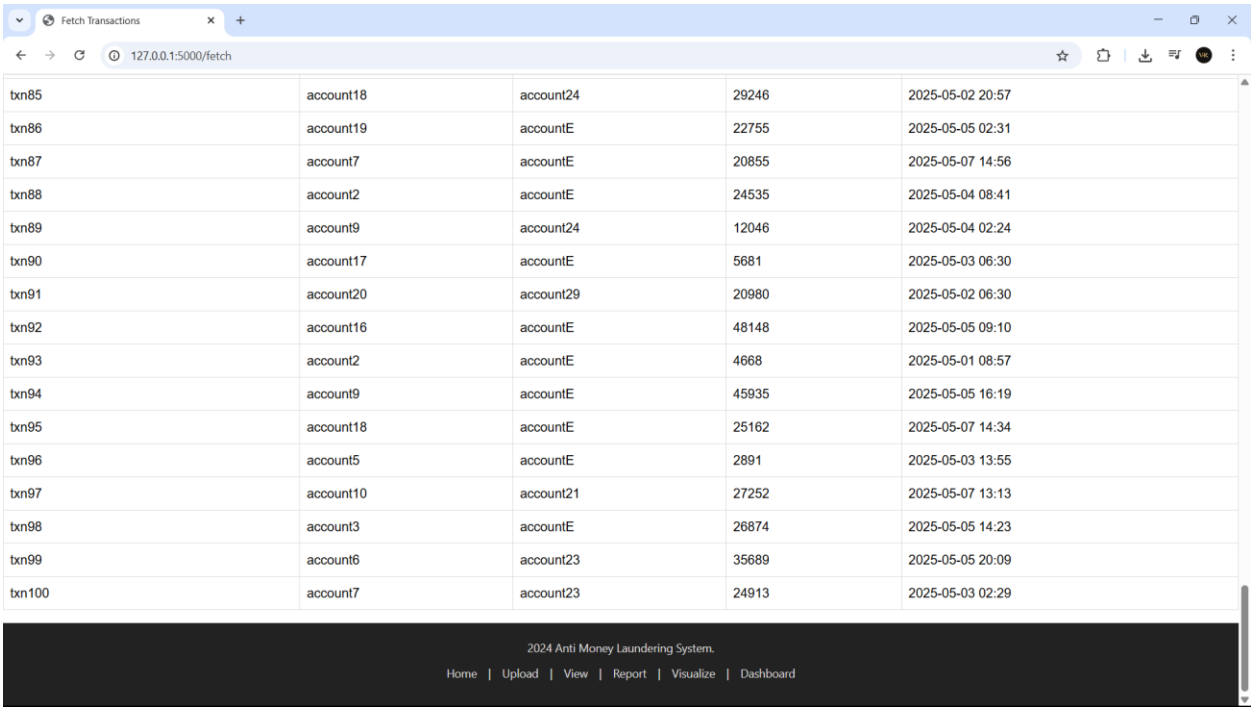

Upload
127.0.0.1:5000/upload
Anti - Money Laundering System
Home
Upload
View
Report
Visualize
Dashboard
Upload CSV File
Please select a CSV file from your device to upload. This file will be processed for anti-money laundering checks.
Choose File
No file chosen
Upload
File Requirements
Format: CSV (Comma-Separated Values)
Size Limit: Maximum file size is 5MB.
Content: Ensure the CSV contains the necessary fields for processing (e.g., Name, Amount, Date).
Tips for Successful Upload
Check that your file is in the correct format before uploading.
Ensure there are no empty rows or columns in your CSV file.
If you encounter any issues, please contact


Upload
127.0.0.1:5000/upload
Upload CSV File
Please select a CSV file from your device to upload. This file will be processed for anti-money laundering checks.
Choose File
No file chosen
Upload
File Requirements
Format: CSV (Comma-Separated Values)
Size Limit: Maximum file size is 5MB.
Content: Ensure the CSV contains the necessary fields for processing (e.g., Name, Amount, Date).
Tips for Successful Upload
Check that your file is in the correct format before uploading.
Ensure there are no empty rows or columns in your CSV file.
If you encounter any issues, please contact support.
2024 Anti Money Laundering System
Home | Upload | View | Report | Visualize | Dashboard

View page

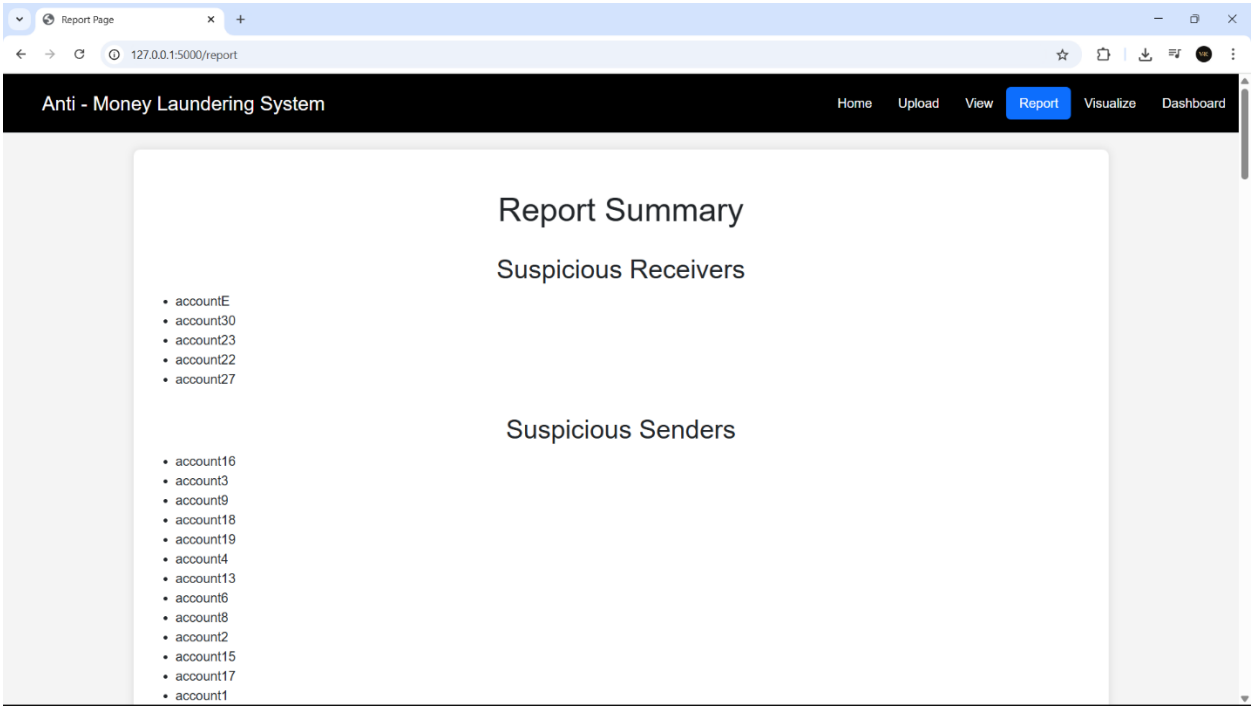


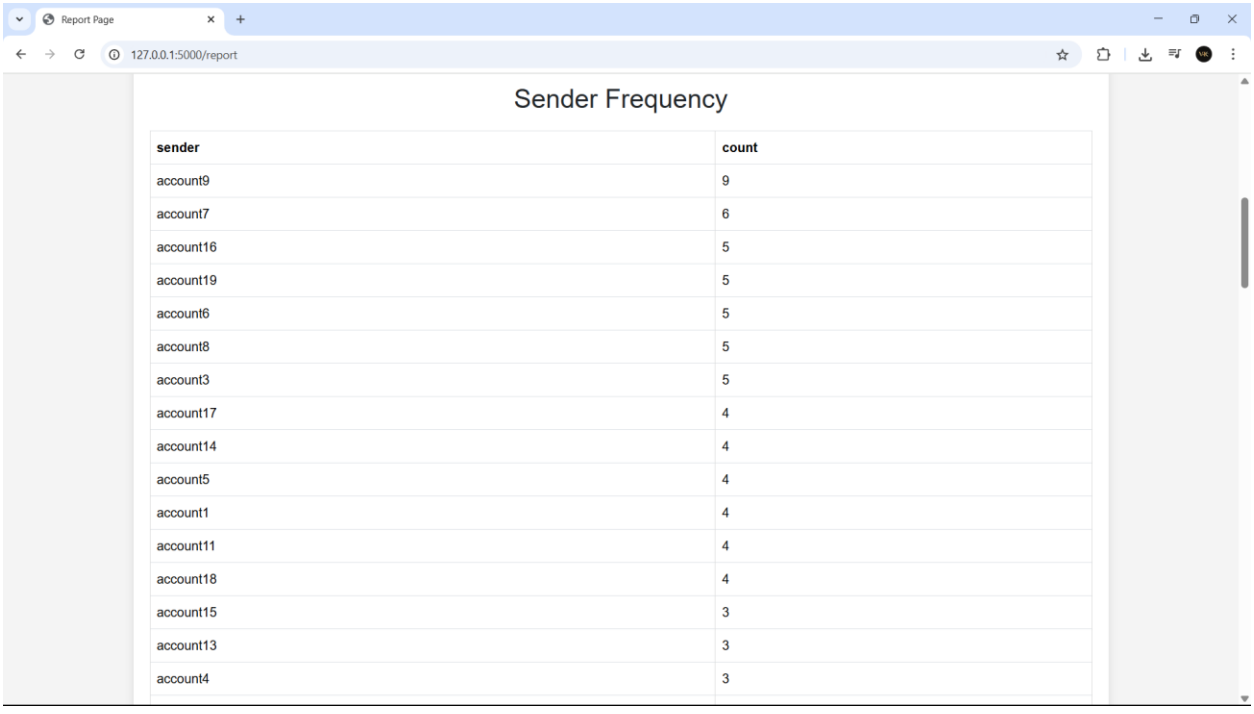
transaction_id	sender	receiver	amount	timestamp
txn1	account10	account29	12795	2025-05-04 05:51
txn2	account16	accountE	39107	2025-05-02 06:31
txn3	account3	account27	13863	2025-05-01 22:05
txn4	account16	account28	21598	2025-05-02 11:08
txn5	account18	account28	35575	2025-05-02 11:45
txn6	account9	account23	34490	2025-05-05 22:14
txn7	account9	accountE	36267	2025-05-01 16:11
txn8	account18	accountE	39305	2025-05-06 20:25
txn9	account16	accountE	30059	2025-05-02 11:06
txn10	account16	accountE	42671	2025-05-02 01:14
txn11	account19	account27	38946	2025-05-05 13:55
txn12	account4	accountE	33793	2025-05-01 13:31
txn13	account13	accountE	23506	2025-05-01 03:12
txn14	account6	accountE	12622	2025-05-01 04:48

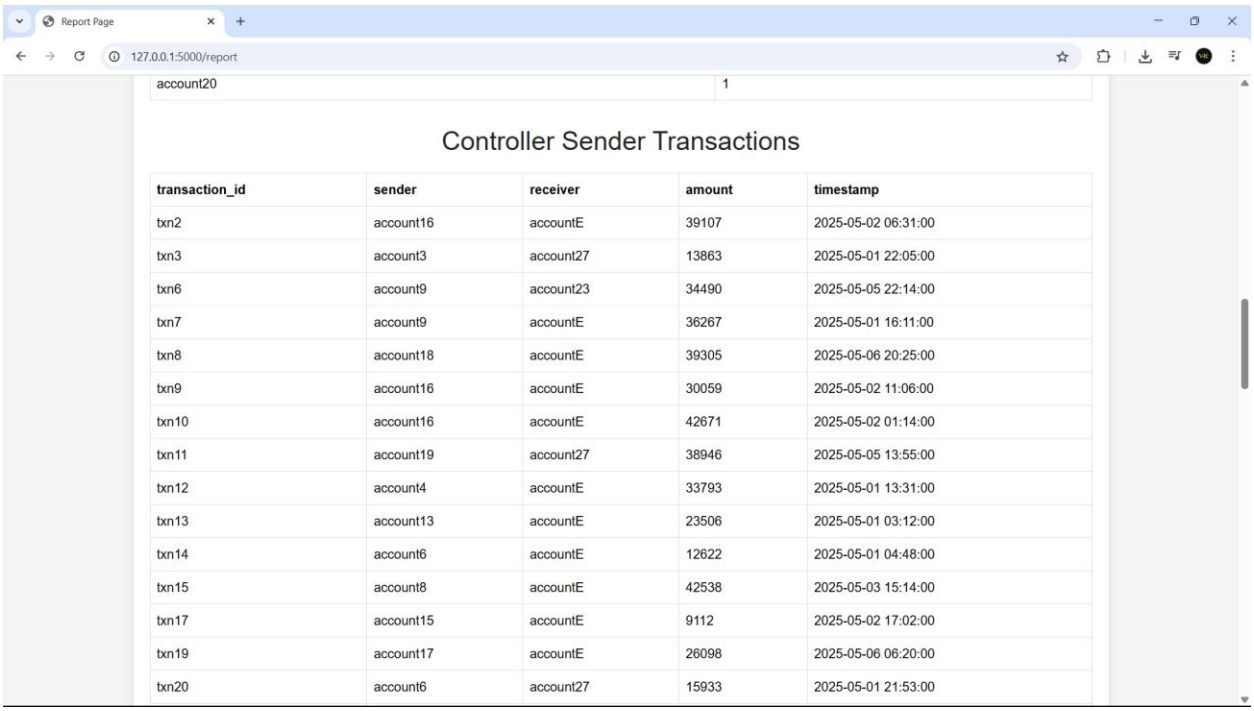


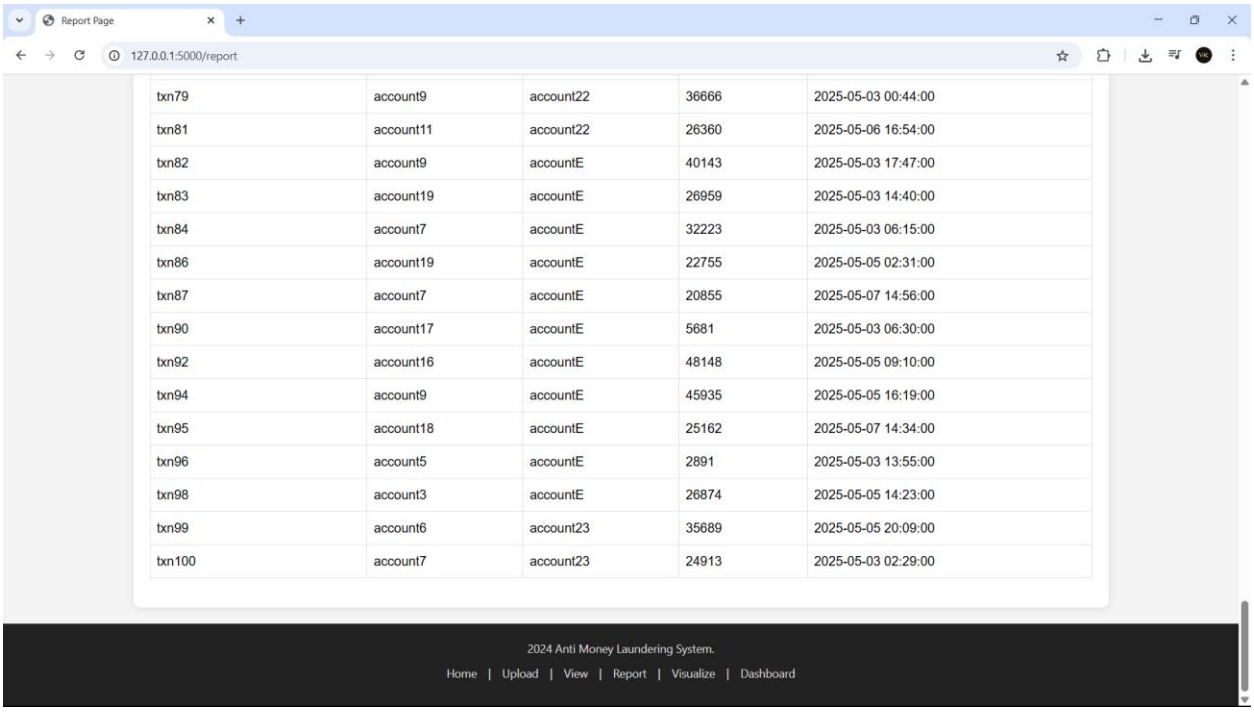
txn85	account18	account24	29246	2025-05-02 20:57
txn86	account19	accountE	22755	2025-05-05 02:31
txn87	account7	accountE	20855	2025-05-07 14:56
txn88	account2	accountE	24535	2025-05-04 08:41
txn89	account9	account24	12046	2025-05-04 02:24
txn90	account17	accountE	5681	2025-05-03 06:30
txn91	account20	account29	20980	2025-05-02 06:30
txn92	account16	accountE	48148	2025-05-05 09:10
txn93	account2	accountE	4668	2025-05-01 08:57
txn94	account9	accountE	45935	2025-05-05 16:19
txn95	account18	accountE	25162	2025-05-07 14:34
txn96	account5	accountE	2891	2025-05-03 13:55
txn97	account10	account21	27252	2025-05-07 13:13
txn98	account3	accountE	26874	2025-05-05 14:23
txn99	account6	account23	35689	2025-05-05 20:09
txn100	account7	account23	24913	2025-05-03 02:29

Report page

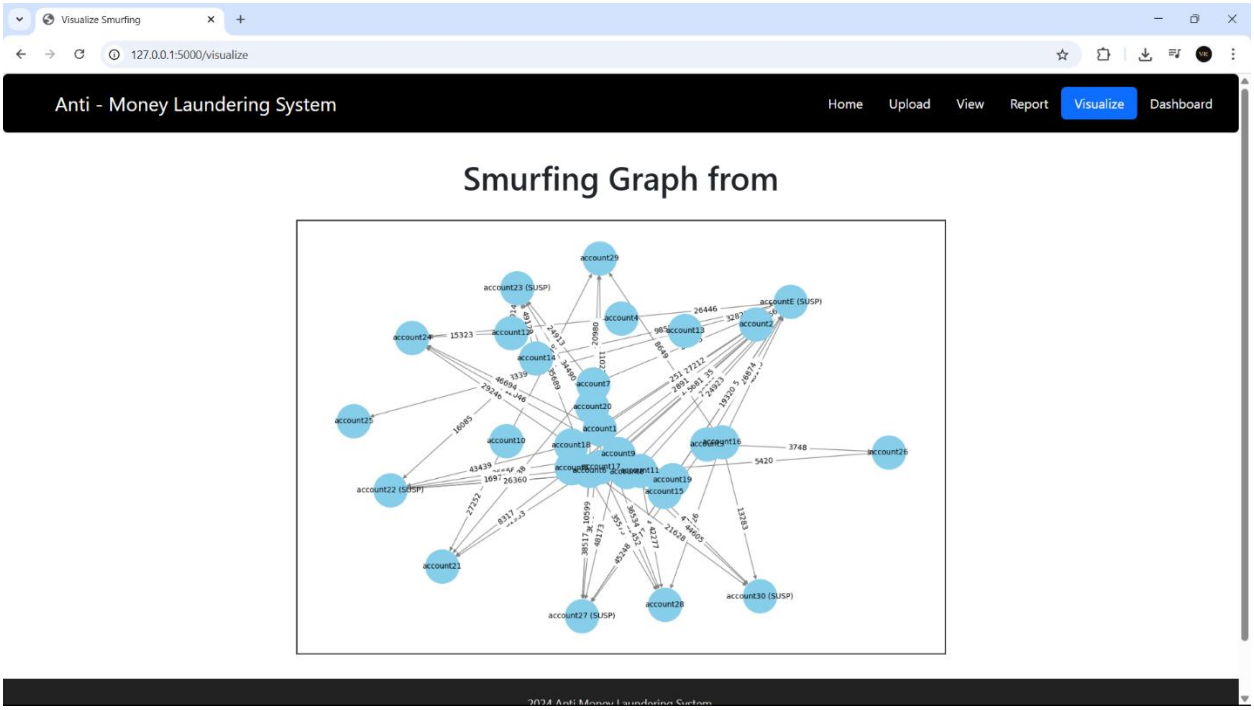

Report Page
127.0.0.1:5000/report
Anti - Money Laundering System
Home
Upload
View
Report
Visualize
Dashboard
Report Summary
Suspicious Receivers
accountE
account30
account23
account22
account27
Suspicious Senders
account16
account3
account9
account18
account19
account4
account13
account6
account8
account2
account15
account17
account1


Report Page
127.0.0.1:5000/report
Sender Frequency
sender
count
account9
9
account7
6
account16
5
account19
5
account6
5
account8
5
account3
5
account17
4
account14
4
account5
4
account1
4
account11
4
account18
4
account15
3
account13
3
account4
3

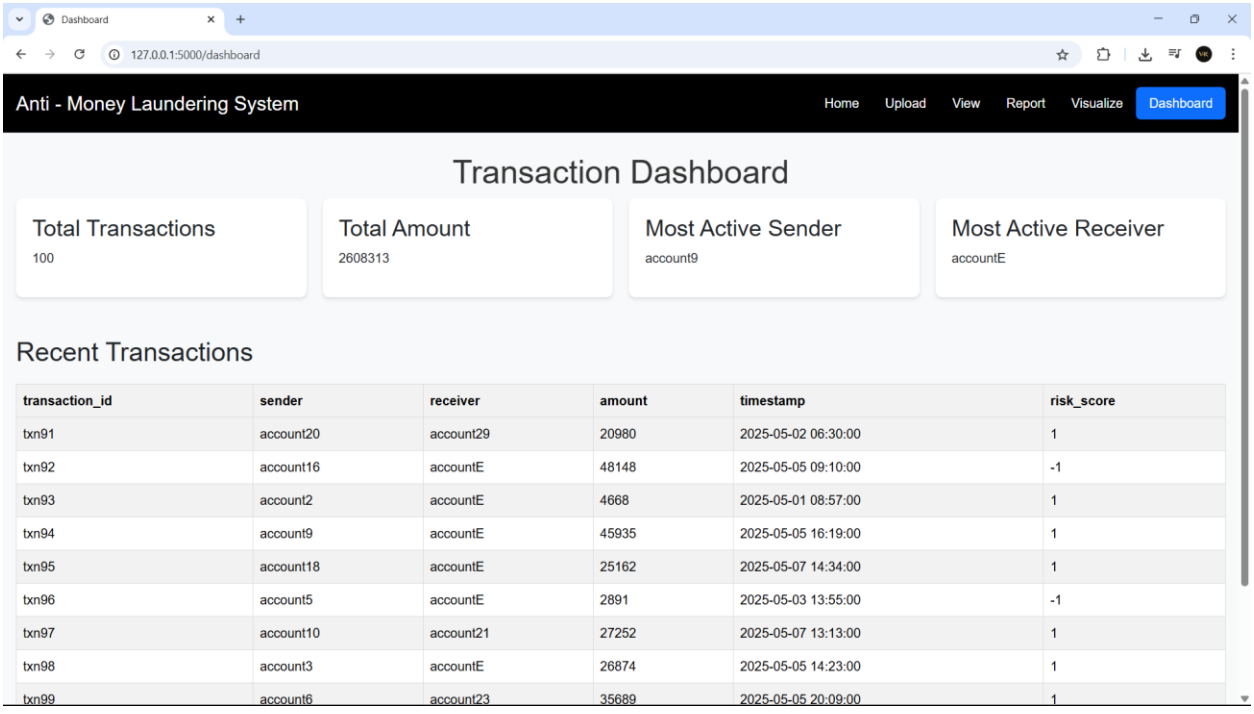

Report Page
127.0.0.1:5000/report
account20 1
Controller Sender Transactions
transaction_id sender receiver amount timestamp
txn2 account16 accountE 39107 2025-05-02 06:31:00
txn3 account3 account27 13863 2025-05-01 22:05:00
txn6 account9 account23 34490 2025-05-05 22:14:00
txn7 account9 accountE 36267 2025-05-01 16:11:00
txn8 account18 accountE 39305 2025-05-06 20:25:00
txn9 account16 accountE 30059 2025-05-02 11:06:00
txn10 account16 accountE 42671 2025-05-02 01:14:00
txn11 account19 account27 38946 2025-05-05 13:55:00
txn12 account4 accountE 33793 2025-05-01 13:31:00
txn13 account13 accountE 23506 2025-05-01 03:12:00
txn14 account6 accountE 12622 2025-05-01 04:48:00
txn15 account8 accountE 42538 2025-05-03 15:14:00
txn17 account15 accountE 9112 2025-05-02 17:02:00
txn19 account17 accountE 26098 2025-05-06 06:20:00
txn20 account6 account27 15933 2025-05-01 21:53:00


Report Page
127.0.0.1:5000/report
txn79 account9 account22 36666 2025-05-03 00:44:00
txn81 account11 account22 26360 2025-05-06 16:54:00
txn82 account9 accountE 40143 2025-05-03 17:47:00
txn83 account19 accountE 26959 2025-05-03 14:40:00
txn84 account7 accountE 32223 2025-05-03 06:15:00
txn86 account19 accountE 22755 2025-05-05 02:31:00
txn87 account7 accountE 20855 2025-05-07 14:56:00
txn90 account17 accountE 5681 2025-05-03 06:30:00
txn92 account16 accountE 48148 2025-05-05 09:10:00
txn94 account9 accountE 45935 2025-05-05 16:19:00
txn95 account18 accountE 25162 2025-05-07 14:34:00
txn96 account5 accountE 2891 2025-05-03 13:55:00
txn98 account3 accountE 26874 2025-05-05 14:23:00
txn99 account6 account23 35689 2025-05-05 20:09:00
txn100 account7 account23 24913 2025-05-03 02:29:00
2024 Anti Money Laundering System.
Home | Upload | View | Report | Visualize | Dashboard

Visualizing page



Dashboard page



transaction_id	sender	receiver	amount	timestamp	risk_score
txn91	account20	account29	20980	2025-05-02 06:30:00	1
txn92	account16	accountE	48148	2025-05-05 09:10:00	-1
txn93	account2	accountE	4668	2025-05-01 08:57:00	1
txn94	account9	accountE	45935	2025-05-05 16:19:00	1
txn95	account18	accountE	25162	2025-05-07 14:34:00	1
txn96	account5	accountE	2891	2025-05-03 13:55:00	-1
txn97	account10	account21	27252	2025-05-07 13:13:00	1
txn98	account3	accountE	26874	2025-05-05 14:23:00	1
txn99	account6	account23	35689	2025-05-05 20:09:00	1

CHAPTER 10

CONCLUSION

The integration of AI, machine learning, and cryptography offers a significant improvement in the ability to detect and prevent money laundering in financial systems. By leveraging AI's pattern recognition capabilities and cryptography's secure transaction management, the proposed system can offer real-time detection and more accurate identification of illicit activities, particularly in complex environments such as cryptocurrency exchanges. This approach addresses the limitations of traditional rule-based systems and offers a scalable, adaptive, and secure solution to combat money laundering.

Future Scope:

- **Enhanced Cryptocurrency Integration**: The system can be expanded to support decentralized exchanges and blockchain-based transactions.
- **More Sophisticated AI Models**: Future versions of the system can integrate more advanced machine learning models such as deep learning for more accurate anomaly detection.
- **Real-Time Global Monitoring**: The system can be scaled to monitor global financial transactions across various platforms and regions, improving its overall effectiveness in preventing cross-border money laundering.
- **User Behavior Analysis**: Adding features to analyze user behavior patterns for better detection of identity theft or synthetic fraud.
- **Continuous Model Training**: Implementing a continuous learning approach where the model updates itself based on new data and evolving laundering techniques.

Applications:

1. **Banking and Financial Institutions**: Detect suspicious activities in traditional financial transactions.
2. **Cryptocurrency Exchanges**: Monitor and prevent illicit activities in cryptocurrency transactions.

3. **Government Regulatory Bodies**: Ensure compliance with AML regulations by detecting illegal financial activities.
4. **Insurance Companies**: Identify fraudulent claims or money laundering through insurance policies.
5. **Cross-Border Transactions**: Improve global financial monitoring for international trade and remittances.

CHAPTER 11

REFERENCES

9.1 REFERENCE:

[1] C. Alexandre and J. Balsa, “A Multi-Agent System Based Approach to Fight Financial Fraud: An Application to Money Laundering,” Preprint, vol. 2017, no. January, p. 14, 2017, doi: 10.20944/preprints201801.0193.v1.

[2] Y. Zhou et al., “Analyzing and Detecting Money-Laundering Accounts in Online Social Networks,” IEEE Netw., vol. 32, no. 3, pp. 115–121, 2018, doi: 10.1109/MNET.2017.1700213.

[3] S. Farrugia, J. Ellul, and G. Azzopardi, “Detection of illicit accounts over the Ethereum blockchain,” Expert Syst. Appl., vol. 150, no. February 2019, p. 113318, 2020, doi: 10.1016/j.eswa.2020.113318.

[4] J. Buchanan and Y. Shen, “To launder or not to launder: are there positive effects for the economies of countries who launder money?,” Account. Financ., 2020, doi: 10.1111/acfi.12680.

[5] D. Savage, Q. Wang, P. Chou, X. Zhang, and X. Yu, “Detection of money laundering groups using supervised learning in networks,” 2016, [Online]. Available: http://arxiv.org/abs/1608.00708.

[6] M. Jullum, A. Løland, R. B. Huseby, G. Ånonsen, and J. Lorentzen, “Detecting money laundering transactions with machine learning,” J. Money Laund. Control, vol. 23, no. 1, pp. 173–186, 2020, doi: 10.1108/JMLC-07-2019-0055.

[7] Y. Zhou et al., “Analyzing and Detecting Money-Laundering Accounts in Online Social Networks.,” IEEE Netw., vol. 32, no. 3, pp. 115–121, 2018, doi: 10.1109/MNET.2017.1700213.

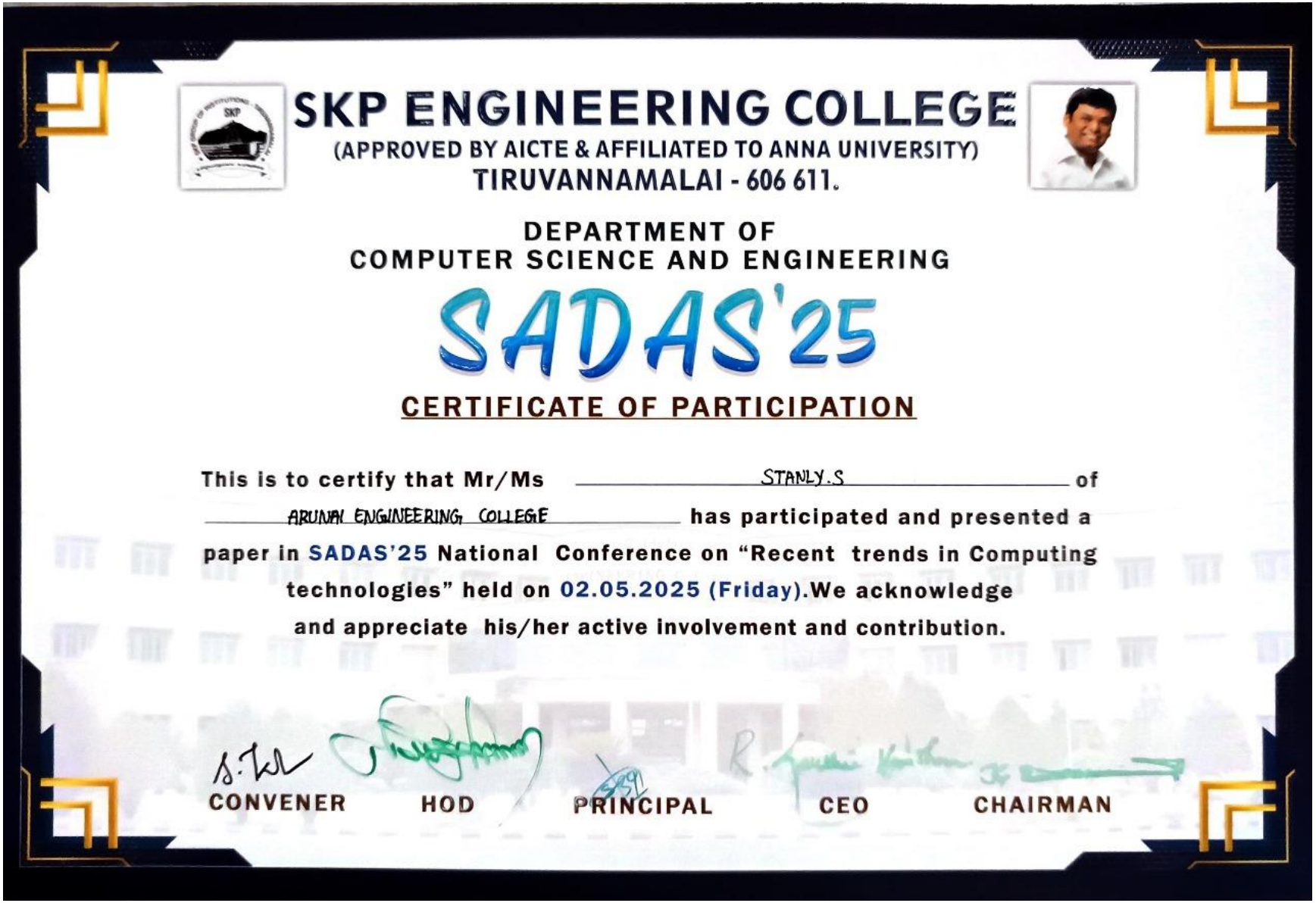
[8] J. A. Álvarez-Jareño, E. Badal-Valero, and J. M. Pavía, “Using machine learning for financial fraud detection in the accounts of companies investigated for money laundering,” Work. Pap., 2017,[Online].Available:http://www.doctreballeco.uji.es/wpficheros/Alvarez_et_al_07_2017.pdf

[9] D. Abdelhamid, S. Khaoula, and O. Atika, “Automatic Bank Fraud Detection Using Support Vector Machines,” Int. Conf. …, pp. 10–17, 2014, [Online]. Available: http://sdiwc.net/digitallibrary/automatic-bank-fraud-detection-using-support-vector-machines.

[10] B. N. Pambudi, I. Hidayah, and S. Fauziati, “Improving Money Laundering Detection Using Optimized Support Vector Machine,” 2019 2nd Int. Semin. Res. Inf. Technol. Intell. Syst. ISRITI 2019, pp. 273–278, 2019, doi: 10.1109/ISRITI48646.2019.9034655.

[11] V. Jayasree and R. V. Siva Balan, “Money laundering regulatory risk evaluation using Bitmap Index-based Decision Tree,” J. Assoc. Arab Univ. Basic Appl. Sci., vol. 23, pp. 96–102, 2017, doi: 10.1016/j.jaubas.2016.03.001.

[12] S. Chakrabarti, Institute of Electrical and Electronics Engineers, and I. Institute of Engineering & Management (Kolkata, “2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON) : 1st-3rd November, 2018, University of British Columbia, Vancouver, Canada,” 2018 IEEE 9th Annu. Inf. Technol. Electron. Mob. Commun. Conf., pp. 37–41, 2018.



SKP ENGINEERING COLLEGE

(APPROVED BY AICTE & AFFILIATED TO ANNA UNIVERSITY)

TIRUVANNAMALAI - 606 611.

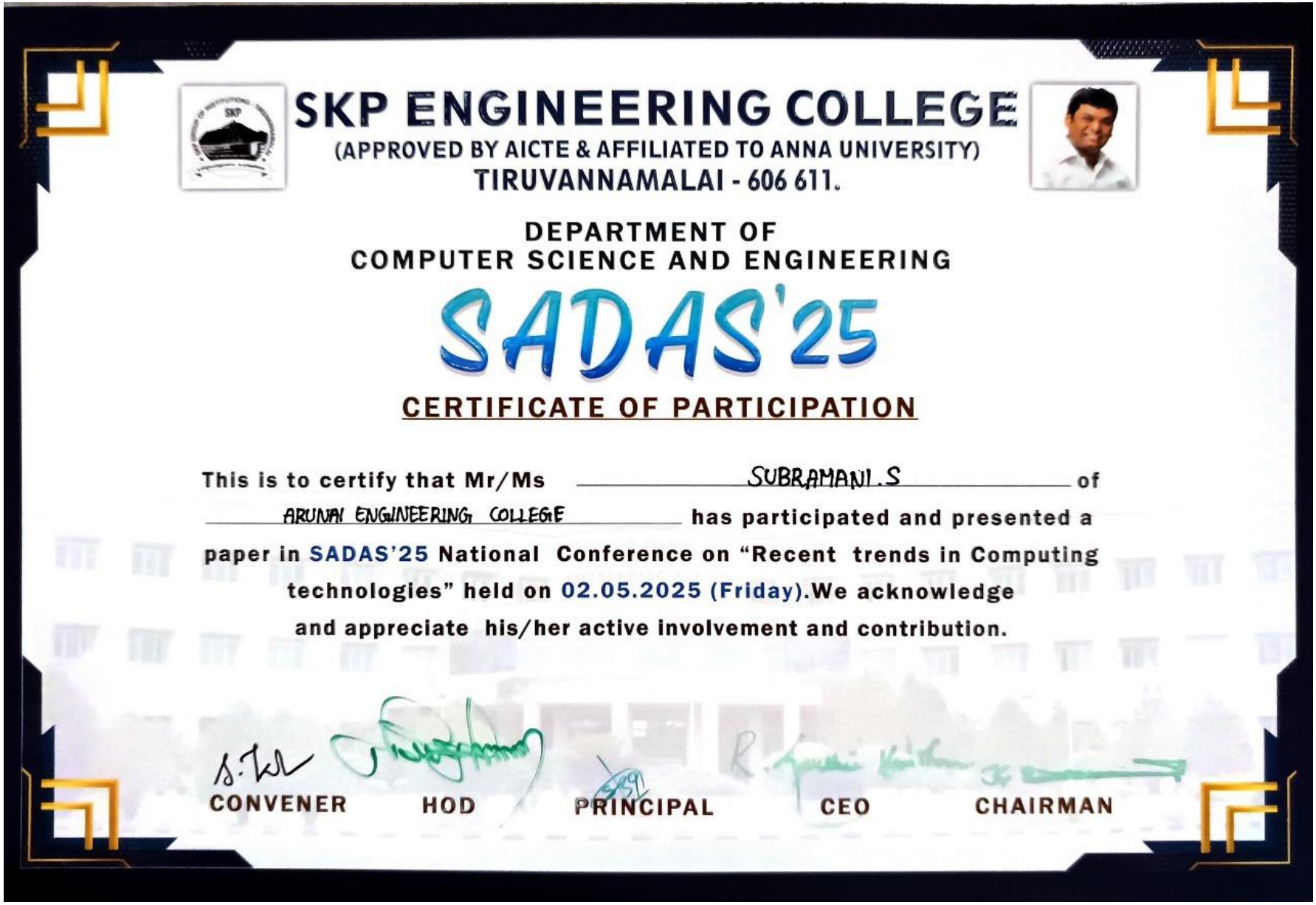
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SADAS'25

CERTIFICATE OF PARTICIPATION

This is to certify that Mr/Ms STANLY.S of ARUNAI ENGINEERING COLLEGE has participated and presented a paper in SADAS'25 National Conference on "Recent trends in Computing technologies" held on 02.05.2025 (Friday).We acknowledge and appreciate his/her active involvement and contribution.

CONVENER HOD PRINCIPAL CEO CHAIRMAN



SKP ENGINEERING COLLEGE

(APPROVED BY AICTE & AFFILIATED TO ANNA UNIVERSITY)

TIRUVANNAMALAI - 606 611.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SADAS'25

CERTIFICATE OF PARTICIPATION

This is to certify that Mr/Ms SUBRAMANI.S of ARUNAI ENGINEERING COLLEGE has participated and presented a paper in SADAS'25 National Conference on "Recent trends in Computing technologies" held on 02.05.2025 (Friday).We acknowledge and appreciate his/her active involvement and contribution.

CONVENER HOD PRINCIPAL CEO CHAIRMAN

SKP ENGINEERING COLLEGE

(APPROVED BY AICTE & AFFILIATED TO ANNA UNIVERSITY)

TIRUVANNAMALAI - 606 611.



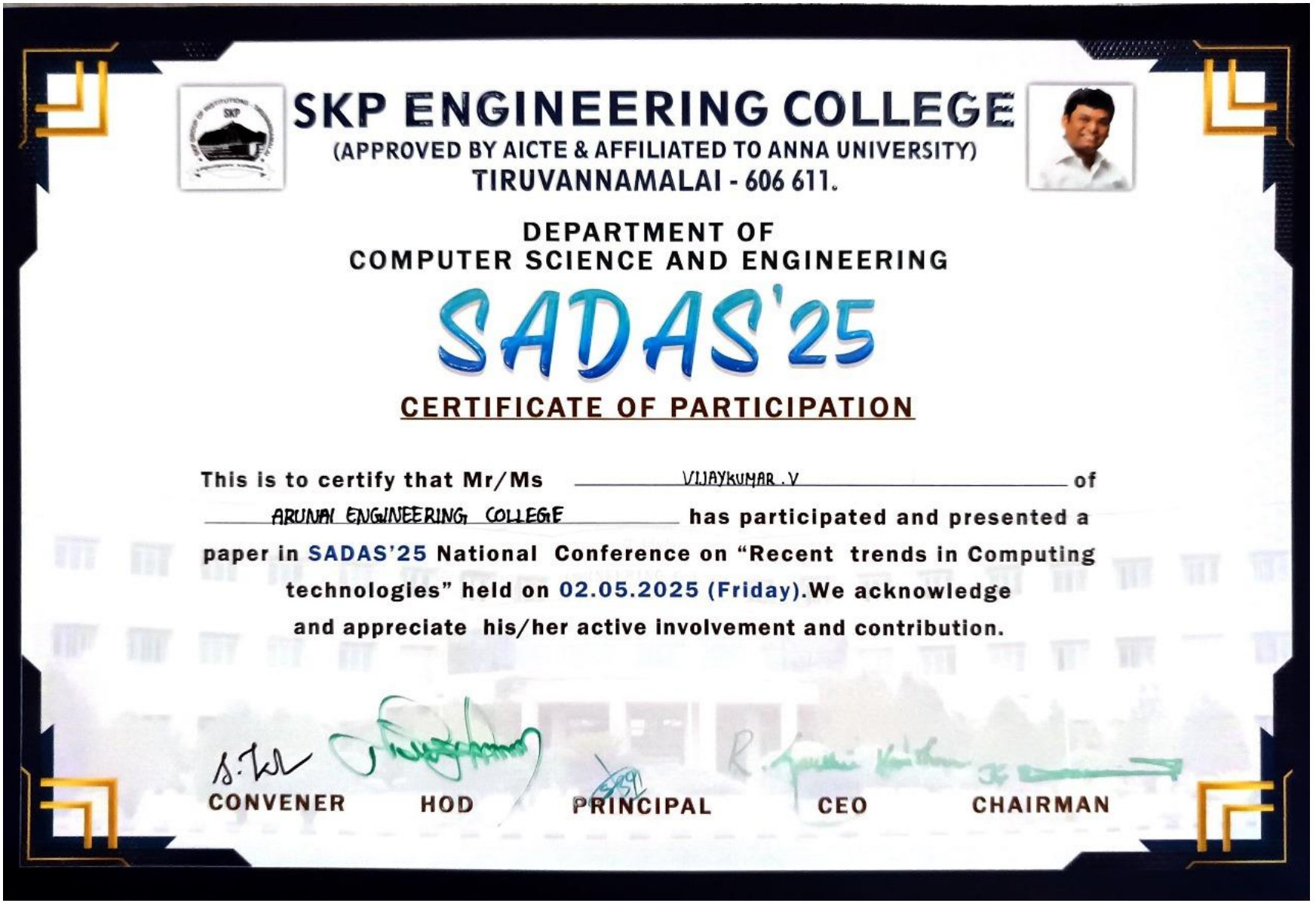
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SADAS'25

CERTIFICATE OF PARTICIPATION

This is to certify that Mr/Ms SUDHARSHAN.S of ARUNAI ENGINEERING COLLEGE has participated and presented a paper in SADAS'25 National Conference on "Recent trends in Computing technologies" held on 02.05.2025 (Friday).We acknowledge and appreciate his/her active involvement and contribution.

CONVENER HOD PRINCIPAL CEO CHAIRMAN



SKP ENGINEERING COLLEGE

(APPROVED BY AICTE & AFFILIATED TO ANNA UNIVERSITY)

TIRUVANNAMALAI - 606 611.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SADAS'25

CERTIFICATE OF PARTICIPATION

This is to certify that Mr/Ms VIJAYKUMAR .V of ARUNAI ENGINEERING COLLEGE has participated and presented a paper in SADAS'25 National Conference on "Recent trends in Computing technologies" held on 02.05.2025 (Friday).We acknowledge and appreciate his/her active involvement and contribution.

CONVENER HOD PRINCIPAL CEO CHAIRMAN

5104 - ARUNAI ENGINEERING COLLEGE, TIRUVANNAMALAI -606 603
(An Autonomous Institution)
Department of Computer Science & Engineering

Batch No. 14 **CS3811 - Project Work Evaluation Form for B.E. (2024-2025)** **Semester: VIII**

Project Details	**Domain / Area:** AIML			
	Title: Anti Money Laundering System in Cryptocurrency using AIML			
Register Number	**Student Name**	**E-mail id**	**Contact number**	**Guided by**
510421104116	V. Vijayakumar	vijayakumarvenkatesan2k4@gmail.com	9344710986	Mrs. Neelamegam
510421104102	S. Sudharshan	sudharshan567401@gmail.com	6381490247	
510421104100	S. Subiyamani	anbashoo1115@gmail.com	8883388527	
510421104098	S. Stanly	stanly5745@gmail.com	7604855455	

Team Members Contribution and Performance (Each category carries 10 points)

Date: 21.2.25

I Review marks

Subject Matter	1 V.V	2 S.	3	4 Vi
Understanding background & Project Goal	10	10	10	10
Knowledge about the existing system	9	9	9	9
Summaries, Algorithms and Project features	9	9	9	9
Project Planning & Technical Design	8	8	8	8
Presentation Skills	8	8	8	8
Total				
Staff Signature	MJ	MJ	MJ	MJ

Date: 27.3.25

II Review marks

Subject Matter	1	2	3	4
Project Planning & Technical Design	10	8	9	9
Summaries of ultimate findings	10	8	9	8
Explanation of Project Modules	10	9	8	9
Presentation Skills	10	8	9	8
Question and Answer	10	9	9	9
Total				
Staff Signature	MJ	MJ	MJ	MJ

Team Members Contribution and Performance (Each category carries 10 points)

Date:

III Review marks

Subject Matter	1	2	3	4
Understanding background & Project Goal	10	10	10	10
Knowledge about the existing system	10	10	10	10
Summaries, Algorithms and Project features	10	10	10	10
Project Planning & Technical Design	10	10	10	10
Presentation Skills	10	10	10	10
Total	50	50	50	50
Staff Signature	NJ	NJ	NJ	NJ

Detail of Conference /Paper published/Certificates received

National level conferance in "Recent trends"

Review	Date	Final Comments by the Supervisor	Rating out of 10
1st Review	21/2/25	Need to improve knowledge about existing system	9
2nd Review	27/3/25	Need to improve communication skills	8
3rd Review	6/5/25	To improve the technical systems	10

13/5/25

Guide | **Member 1** | **Head of the Department**

K.Neelamegam

(Name, Designation and Signature)

K.Neelamegam

N. Anandakumar

(N. Anandakumar)